

De la Simulación al Vuelo Real Conectado a la Nube

Guía didáctica de la arquitectura de teleoperación de drones

Sistema UAV de Búsqueda y Rescate (SAR) — Edge Companion en Raspberry Pi 5 y backend en AWS EC2

Pensado para explicarse desde cero, sin dar por conocido el software de simulación de drones

Control de versiones

Autor	Fecha	Versión	Comentarios
Nerea Gorostidi García	25/08/2026	0.99	Borrador inicial

Introducción

Este documento describe el recorrido completo seguido en el proyecto para pasar de una simulación en el ordenador a un dron real, con Pixhawk físico, volando de forma autónoma y supervisado desde la nube. Se organiza en ocho etapas — dos de preparación (Estudio del Arte e Infraestructura) y seis fases de desarrollo — que se documentan en orden, cada una resolviendo una limitación concreta de la anterior y añadiendo una sola pieza nueva al sistema.

Objetivo del documento

Dejar constancia, de forma reproducible, de la arquitectura seguida y de las decisiones tomadas en cada etapa — no solo qué se hizo, sino por qué se hizo así y qué alternativas se consideraron. Sirve como referencia técnica del proyecto y como documento de acompañamiento de la memoria del TFG.

Documentos relacionados

Este documento remite, para el detalle técnico de piezas concretas, a otros tres documentos del mismo proyecto:

Documento	Relación / comentarios
DronSAR_SITL_a_real_TELEM3_mavlink-router.pdf	Guía técnica paso a paso de la conexión por TELEM3, la instalación de mavlink-router, la identificación SYSID/COMPID y los scripts de prueba de la conexión serie — detalle completo de la Fase 5 y los Apéndices B y C de este documento.
MAVLink_Explicado.pdf	Ampliación a nivel de sockets y configuración de por qué hace falta un MAVLink Bridge — detalle completo de la Fase 2 de este documento.
Problematica_transmision_video.pdf	Estrategia de captura, almacenamiento y streaming de vídeo a bordo del dron — detalle completo de una parte de la Fase 6 de este documento.

El código completo de los procesos del nodo edge (receptor.py, vuelo.py, deteccion.py, y los scripts de prueba mencionados en los Apéndices B y C) está publicado en el repositorio

<https://github.com/nereagorostidi/drone-edge-companion>. Los fragmentos de código incluidos en este documento y en los relacionados son extractos con fines explicativos; el repositorio es la fuente canónica y actualizada.

1. Visión general: el recorrido completo

Antes de entrar en el detalle, conviene ver el mapa completo. Este documento explica ocho etapas: dos etapas de preparación (entender el problema y montar la infraestructura mínima) y seis fases de desarrollo que van añadiendo, una a una, las piezas necesarias hasta llegar a un sistema completo, supervisado y controlado a distancia desde internet.

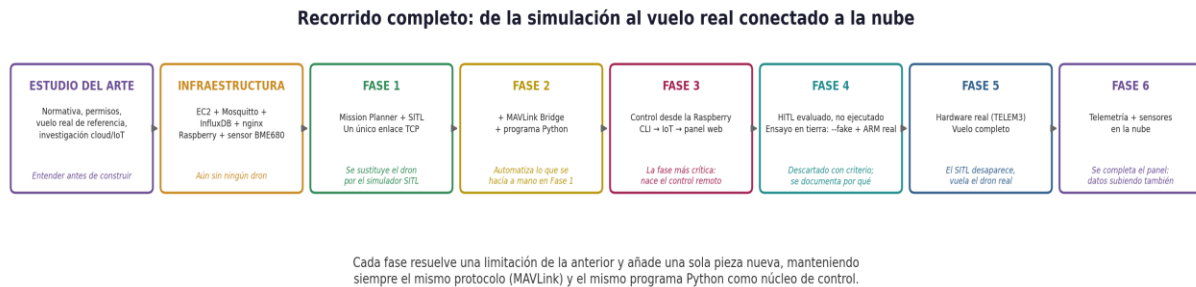


Figura 1. Las ocho etapas del proyecto: cada una añade una sola pieza nueva sobre la anterior.

La idea que hilvana todo el documento es esta: el protocolo de comunicación (MAVLink) y la lógica de control en Python no cambian nunca. Lo único que va cambiando, etapa a etapa, es dónde vive físicamente cada pieza y cómo se conectan entre sí.

2. Estudio del Arte

Antes de escribir una sola línea de código o de montar cualquier infraestructura, esta etapa se dedica a entender bien el problema: cómo funciona realmente un dron, qué normativa aplica, dónde se puede volar, y qué tecnologías existen para la parte de nube e IoT que se usarán más adelante. Es la etapa que menos código produce y más condiciona todo lo que viene después.

2.1 Documentación y normativa: permisos y dónde volar

Antes del primer vuelo real hace falta resolver varias cuestiones administrativas: qué normativa de drones aplica (en España y la UE, el marco EASA por categorías de operación: abierta, específica y certificada, según el peso del dron y el tipo de vuelo), qué permisos o registros son necesarios según esa categoría, y dónde está permitido volar. En esta etapa se identifica una zona adecuada —alejada de aeródromos, zonas urbanas densas o espacios restringidos— y se comprueba con la app oficial correspondiente (por ejemplo, Drone Space de AESA) que el vuelo previsto es legal.

2.2 Un vuelo real, para entender antes de simular

Con los permisos y la ubicación resueltos, esta etapa incluye un primer vuelo real con un dron (no necesariamente el vehículo final del proyecto). El objetivo no es la misión en sí, sino entender de primera mano los conceptos que después se van a reproducir en el ordenador: cómo se comporta la GCS en la práctica, qué se siente pilotar con el mando RC, cuánto tarda en fijar el GPS, cómo responde el dron al viento, qué autonomía real tiene la batería, etc. Es la diferencia entre leer sobre un radioenlace de telemetría y haber visto uno funcionando.

Aprovechando ese vuelo, conviene registrar todo lo que luego pueda servir de referencia para las fases de simulación: el track GPS completo (posición, altura y velocidad en cada instante), y si el dron lleva cámara, el vídeo grabado. Ese mismo patrón —volar primero con un sistema ya fiable y reutilizar sus datos como referencia— es exactamente el que se retoma más adelante, con más detalle, en el apéndice sobre el DJI Neo 2.

2.3 Investigar la infraestructura cloud e IoT a usar

La última pieza de esta etapa es puramente de documentación técnica, de cara a la fase de Infraestructura que viene justo después: entender qué es AWS y cómo se organiza un entorno mínimo en la nube (una VPC, una instancia EC2 y su dimensionamiento), qué papel juega un broker MQTT como Mosquitto en una arquitectura IoT, y qué es una base de datos de series temporales como InfluxDB y por qué encaja mejor que una base de datos relacional clásica para este tipo de datos (lecturas repetidas en el tiempo, con marca temporal, que rara vez se actualizan una vez escritas).

2.4 Así funciona un dron real

Con esa base ya sentada, esta sección explica el sistema real que el resto del documento va a reproducir en el ordenador. Un dron real, en vuelo, no necesita ni ordenador ni programas de simulación: puede comunicarse con tierra de dos formas distintas y, normalmente, simultáneas.

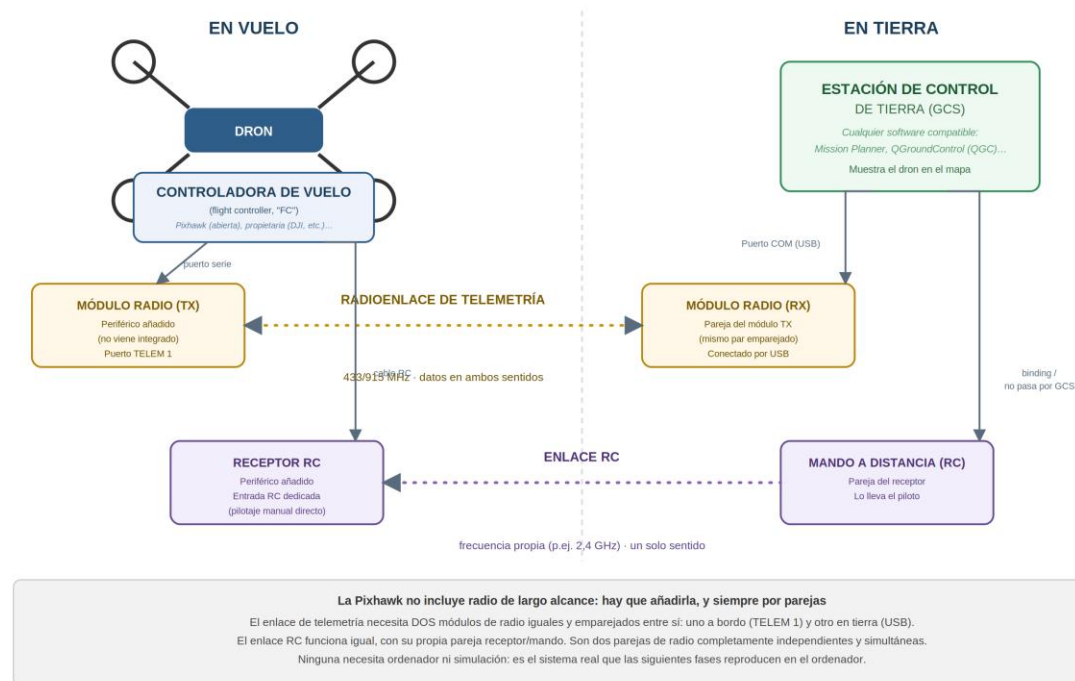


Figura 2. Los dos enlaces de radio reales de un dron: telemetría hacia la GCS, y control manual mediante un mando RC.

2.4.1 El enlace de telemetría, hacia la estación de tierra (GCS)

Es importante aclarar algo que suele generar dudas: la Pixhawk no incluye, de fábrica, ningún radio de largo alcance. El enlace de telemetría es un periférico que hay que añadir aparte, y siempre se compra o se configura por parejas: dos módulos de radio idénticos (o compatibles entre sí) que quedan emparejados. Uno se instala a bordo, conectado al puerto TELEM 1 de la Pixhawk; el otro se conecta por USB al ordenador de tierra. Sin esa pareja completa —los dos módulos, emparejados entre sí— no hay enlace de telemetría posible.

La Pixhawk envía sus datos —posición, batería, estado— por su módulo de radio (habitualmente a 433 o 915 MHz), que el módulo gemelo en tierra recibe y entrega, por USB, a un ordenador. Es importante aclarar también que Mission Planner es solo una de las estaciones de tierra (GCS) posibles: cualquier software compatible con el protocolo MAVLink sirve para este papel, como QGroundControl (QGC), que es otra alternativa muy usada, especialmente fuera del ecosistema ArduPilot.

2.4.2 El enlace RC, para el pilotaje manual directo

De forma independiente, el dron lleva también un receptor RC —de nuevo, un periférico añadido, no integrado en la Pixhawk— conectado directamente a su entrada de radiocontrol. Ese receptor forma pareja con un mando a distancia en manos del piloto, emparejados entre sí igual que el par de módulos de telemetría, pero sin relación alguna con ellos: son dos parejas de radio completamente distintas. Este enlace suele operar en una frecuencia distinta a la de telemetría (por ejemplo 2,4 GHz) y sirve para pilotar el dron en tiempo real, sin pasar por ningún ordenador ni por la GCS.

Idea clave de esta sección

"Telemetría" no es ningún tipo de simulación: es un enlace de radiofrecuencia real, igual que el del mando RC, solo que con otro propósito (supervisar y, en su caso, dar órdenes de alto nivel, en lugar de pilotar directamente). Todo lo que viene a continuación —infraestructura, simulador, Bridge, Raspberry Pi, nube— existe únicamente para desarrollar y probar el software de la GCS y del control automático de forma segura, antes de confiar en estos enlaces reales.

Nota: para no complicar en exceso esta introducción, dejamos de momento fuera del esquema el enlace de vídeo (la imagen de la cámara). Es un tercer enlace independiente de los dos anteriores, que puede resolverse de formas muy distintas según el sistema —streaming por TCP/IP sobre Wi-Fi, un enlace de radio analógico o digital dedicado, etc.— y que retomaremos más adelante, en el apéndice sobre el DJI Neo 2.

3. Infraestructura

Antes de escribir ningún código relacionado con el dron, esta etapa monta la infraestructura mínima que todas las fases posteriores van a reutilizar tal cual: el servidor en la nube, y el primer código en la Raspberry Pi. Deliberadamente, en esta etapa no aparece todavía ningún componente de navegación del dron: el objetivo es validar que el "tubo" de datos entre el borde (edge) y la nube funciona, usando algo mucho más simple de verificar que el vuelo — la lectura de un sensor ambiental.

3.1 El servidor en la nube

En AWS, dentro de una VPC (Virtual Private Cloud, la red privada y aislada donde vive todo lo demás), se levanta una única instancia EC2 de tamaño t3.small (2 vCPU, 2 GB de RAM), suficiente para el volumen de datos de este proyecto. Sobre esa misma instancia se instalan tres servicios, que quedarán ya preparados para que las fases siguientes se limiten a añadir lógica encima, sin tener que montar nada nuevo:

- Mosquitto: el broker MQTT que recibirá, más adelante, tanto los datos de sensores y telemetría (Fase 6) como los comandos de control (Fase 3).
- InfluxDB: la base de datos de series temporales donde se almacenan las lecturas, ya justificada en el Estudio del Arte.
- nginx: un servidor web, instalado y operativo desde ya, aunque todavía sin contenido — se usará para servir el panel de control en fases posteriores.

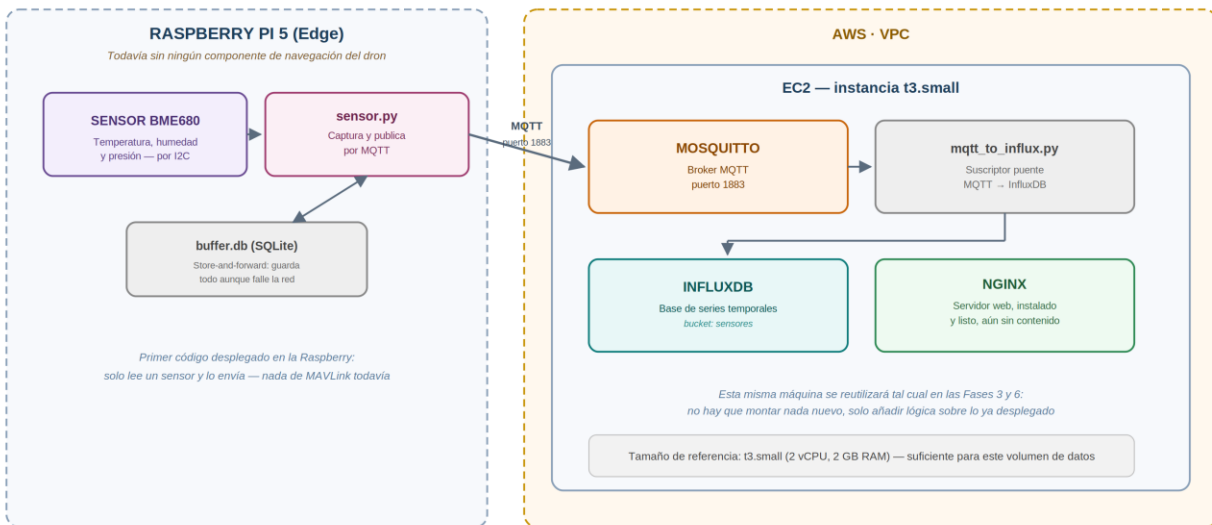


Figura 3. La infraestructura mínima: EC2 t3.small en una VPC, con Mosquitto, InfluxDB y nginx ya desplegados, y la Raspberry Pi enviando sus primeras lecturas por MQTT — todavía sin ningún componente de navegación del dron.

3.2 La Raspberry Pi y el primer canal IoT

En paralelo, se configura la Raspberry Pi 5 que hará de nodo edge en el resto del proyecto, y se conecta por primera vez con la nube: un script (sensor.py) lee un sensor ambiental BME680 por I2C y publica cada lectura

por MQTT hacia el Mosquitto recién desplegado, con un buffer local en SQLite que garantiza que ningún dato se pierde aunque falle la red (arquitectura store-and-forward, ya detallada en la Fase 6 de este documento).

Es intencionado que este primer código no incluya nada de navegación: ni MAVLink, ni Pixhawk, ni Bridge. El objetivo de esta etapa es exclusivamente validar el canal edge-nube con el caso más simple posible, para poder dar por buena la infraestructura antes de complicarla con el control del dron.

4. Fase 1 — La GCS y el simulador, conectados directamente

El primer paso para desarrollar software sin volar un dron de verdad es sustituir la Pixhawk física por un programa que se comporta exactamente igual: el simulador ArduPilot SITL (Software In The Loop). No es una app gráfica; es un proceso que calcula, en tiempo real, la física de vuelo, el GPS y la batería, como si fuera una Pixhawk real.

Lo que ocurre en esta fase es, en el fondo, muy sencillo: se sustituye el radioenlace de telemetría del esquema anterior por una conexión TCP dentro del mismo ordenador. La GCS (Mission Planner o QGC) no nota ninguna diferencia relevante, porque el SITL habla MAVLink de forma nativa —el mismo protocolo que hablaba la Pixhawk real por radio—; no hace falta ningún tipo de traducción. Solo cambia el medio de transporte: antes, radiofrecuencia por el aire; ahora, un socket TCP local.

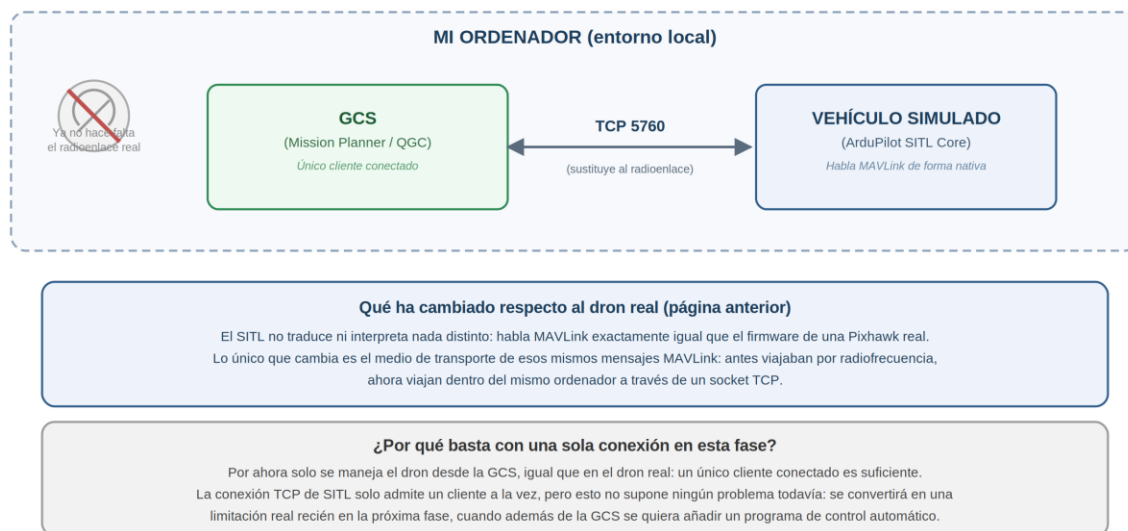


Figura 4. Se sustituye el radioenlace de telemetría por una conexión TCP local; el SITL entiende MAVLink de forma nativa, igual que la Pixhawk real.

En esta fase todavía no hay ninguna complicación de la arquitectura, ni elementos a configurar para que nuestros scripts en Python que interactuaran con el dron puedan funcionar con un dron real o uno simulado. La GCS se conecta directamente al único puerto TCP que abre el simulador (el 5760), exactamente igual que antes se conectaba al módulo de radio, haciendo la conexión por red, sin necesidad de hardware o dispositivos adicionales. De momento, con un único cliente conectado, es perfectamente suficiente: el enlace de telemetría real tampoco tenía que atender a más de una GCS a la vez. La restricción de "un solo cliente" de TCP solo se convertirá en un problema real en la siguiente fase, cuando además de la GCS se quiera incorporar un programa de control automático (nuestros scripts en Python de control).

El objetivo práctico de esta fase es usar únicamente las herramientas ya integradas en la GCS —su función "Simulation" (que lanza el propio SITL por debajo, sin que haga falta tocar la línea de comandos) y su editor de misiones "Plan"— para intentar navegar por los mismos puntos que se sobrevolaron en el vuelo real del Estudio del Arte. Sin escribir todavía ni una línea de Python, esto ya permite comparar cómo se comporta el simulador frente al vuelo real registrado, incluso enviando ordenes al dron a través del propio GCS.

5. Fase 2 — Arquitectura de conexión

Esta fase establece los cimientos de la arquitectura del proyecto resolviendo dos necesidades simultáneas: la concurrencia en el control local del vehículo y el despliegue del entorno cloud mínimo para la ingesta de datos y teleoperación.

A. Arquitectura de Transporte Local: El MAVLink Bridge

Esta fase resuelve un problema de arquitectura que condiciona todo lo que viene después: cómo repartir un enlace exclusivo entre varios consumidores a la vez. El patrón que se establece aquí —un proceso intermedio que concentra el enlace único y lo redistribuye— es el mismo que reaparece, sin cambios de fondo, en la Fase 3 (control remoto) y en la Fase 5 (hardware real); solo cambia la herramienta concreta que lo implementa en cada caso.

El objetivo del proyecto no es solo visualizar el dron en la GCS, sino controlarlo automáticamente mediante un script de Python (por ejemplo, para que despegue, siga una ruta y aterrice sin intervención manual). Pero aquí nos encontramos con un problema, el puerto TCP 5760 del simulador solo admite un cliente conectado a la vez: si la GCS ya lo está usando, el script de Python no puede conectarse también.

Por ello, en esta fase, instalaremos los elementos de arquitectura necesarias para las primeras pruebas y test con nuestros scripts en Python que controlaran el dron, y que ejecutaremos en la misma estación Windows donde esta Mission Planner, con una conexión local (loopback), sin necesidad aun de introducir la raspberry, y de forma que podamos hacer nuestras primeras pruebas en Python sin necesidad de ningún hardware adicional más que el propio portátil donde corremos Mission Planner.

La solución es introducir un MAVLink Bridge, funcionalidad que viene con el propio Mission Planner (una opción de configuración). Es importante dejar clara su función exacta: el Bridge no traduce ni interpreta ningún comando, y no añade ninguna capa de "inteligencia" sobre el protocolo. Es un simple repetidor que copia cada mensaje MAVLink que recibe por un puerto y lo reenvía a todos los demás puertos conectados. Su única función es resolver el problema del cliente único: tomar la conexión TCP exclusiva del simulador y repartirla en varios puertos UDP independientes, que sí admiten varios clientes a la vez.

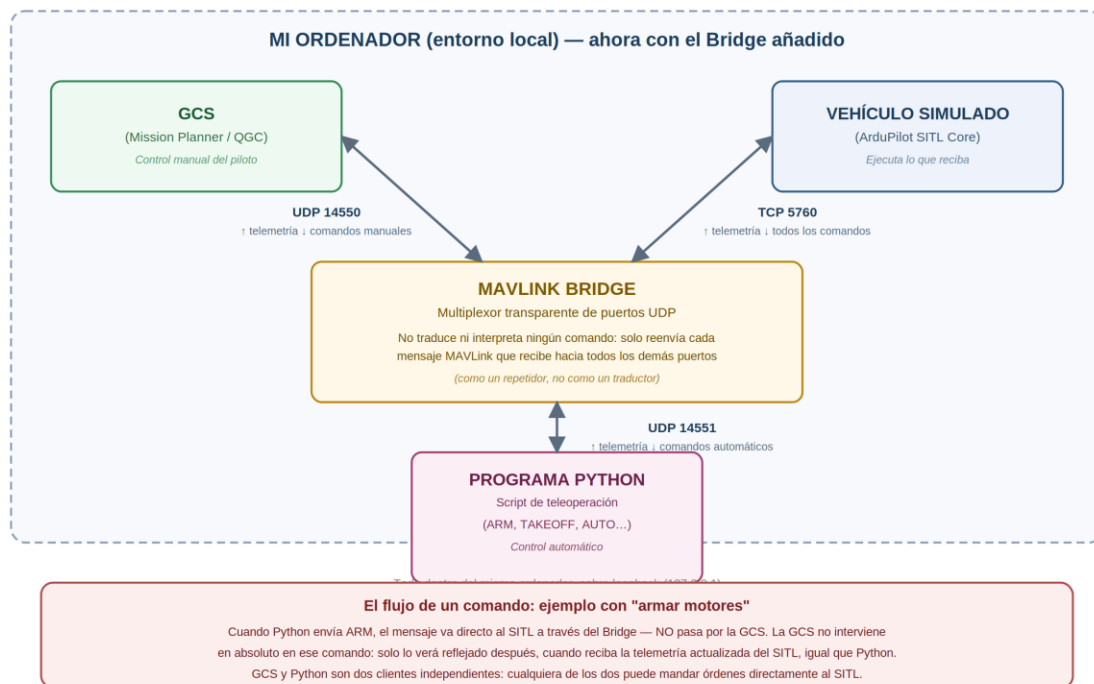


Figura 5. El Bridge reparte, sin traducir nada, la única conexión TCP del simulador entre la GCS y el script de Python.

Nota: el detalle práctico de esta configuración — cómo abrir cada puerto en modo cliente o servidor, la diferencia entre `bind()` y `connect()`, 127.0.0.1 frente a 0.0.0.0, y un caso real de diagnóstico paso a paso con dos scripts de Python (`arm_takeoff.py` y `telemetry.py`) — se explica en profundidad, con diagramas propios, en [MAVLink_Explicado.docx](#).

El flujo de mensajes, paso a paso

Este es el punto que más suele generar confusión, así que conviene explicarlo con un ejemplo concreto: cuando el script de Python envía un comando de armado (ARM), ese mensaje viaja directamente desde Python hasta el SITL a través del Bridge. No pasa por la GCS en ningún momento. La GCS es un cliente totalmente independiente: solo se entera de que el dron se ha armado cuando, un instante después, recibe la telemetría actualizada que el SITL envía —por el Bridge— a todos los clientes conectados.

En otras palabras, tanto la GCS como Python pueden enviar órdenes directamente al SITL, cada uno por su propio puerto UDP, y ambos reciben la misma telemetría de vuelta. Es exactamente el mismo esquema "varios clientes, un solo vehículo" que ya existía de forma simple en el punto de partida (GCS + mando RC, cada uno con su propio enlace hacia el dron), solo que ahora ambos canales pasan por el mismo Bridge en vez de por dos radioenlaces distintos.

Una aclaración importante sobre ese último punto: un GCS como Mission Planner nunca es un simple "visor" de telemetría. Es, en sí mismo, un participante MAVLink completo y bidireccional — cada vez que un piloto pulsa un botón en su interfaz (armar, cambiar de modo, RTL), está generando y enviando un comando hacia el vehículo, exactamente igual que lo hace un script por código. La telemetría (dron → GCS) es solo la mitad

"de subida" de ese enlace; la mitad "de bajada" (comandos GCS → dron) existe siempre, la use un humano o un programa.

Esto explica por qué el ejemplo anterior dice que el comando de armado del script "no pasa por la GCS": se refiere a que no pasa por su lógica de interfaz ni requiere ninguna decisión humana, no a que el proceso de Mission Planner quede fuera del camino. En el montaje concreto de este proyecto (donde el propio Mission Planner hace también de Bridge, en vez de un MAVLink Proxy independiente — ver más abajo), los comandos del script SÍ atraviesan físicamente el mismo proceso de Mission Planner, mediante su función de reenvío, antes de llegar al SITL. Con un MAVLink Proxy separado (como mavlink-router), en cambio, el Bridge es un proceso distinto y el comando nunca pasa por el proceso de la GCS en absoluto; el resultado práctico es el mismo (ambos caminos reciben la misma telemetría), pero conviene tener clara la diferencia.

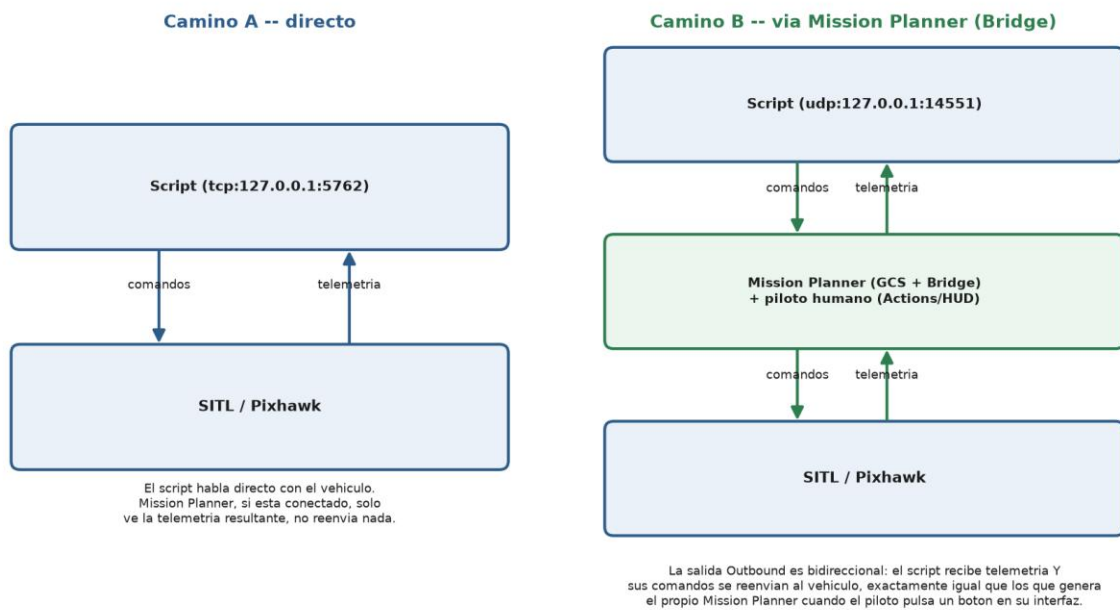


Ilustración: los dos caminos posibles para los comandos de un script — directo al SITL, o a través del GCS/Bridge, que siempre reenvía en ambos sentidos. Ver la Figura 5 y la sección 5.4 de MAVLink_Explicado.docx para el detalle técnico completo, con el caso concreto de arm_takeoff.py y telemetry.py.

Puertos de esta fase

- TCP 5760: puerto nativo del simulador SITL (solo lo usa el Bridge).
- UDP 14550: puerto por el que el Bridge reenvía los datos hacia y desde la GCS.
- UDP 14551: puerto por el que el Bridge reenvía los datos hacia y desde el script de Python.
- Todo ocurre dentro del mismo ordenador, sobre la interfaz de loopback (127.0.0.1): no hay tráfico de red real todavía.

MAVLink Router: la alternativa estándar y portátil al Bridge

La función de "Bridge" que cumple aquí Mission Planner — copiar cada mensaje MAVLink recibido por un puerto y reenviarlo a todos los demás puertos conectados — no es exclusiva suya: es un patrón estándar dentro del ecosistema MAVLink, con herramientas dedicadas que hacen exactamente lo mismo sin depender de ninguna GCS gráfica. La más extendida es mavlink-router (también hay una herramienta conocida como mavproxy, de una generación anterior, que hace lo mismo), un proceso ligero de línea de comandos que abre un endpoint de entrada (un puerto serie, TCP o UDP) y lo redistribuye hacia tantos endpoints de salida como se necesiten.

La diferencia práctica con el Bridge de Mission Planner es que un MAVLink Router no necesita ninguna interfaz gráfica ni un PC dedicado: puede correr en segundo plano en la propia Raspberry Pi, sin que Mission Planner tenga que estar abierto en ningún sitio. Esto lo hace especialmente útil en dos momentos del recorrido de este documento: en la Fase 3, como sustituto del Bridge alojado en el PC (la Raspberry podría correr su propio proxy en vez de depender de que el PC con Mission Planner siga encendido y conectado por Wi-Fi); y en las Fases 5/6, cuando la Raspberry requiera repartir la telemetría real del Pixhawk — que llegará por un único puerto serie exclusivo — entre más de un programa local a la vez: exactamente el mismo problema del "puerto único" que resuelve el Bridge en la Fase 2, ahora aplicado al hardware real.

Un MAVLink Router (o Proxy) nos aporta lo siguiente frente al Bridge de Mission Planner:

- No depende de tener una GCS gráfica abierta: es un proceso de consola, ideal para un companion computer sin pantalla, como la Raspberry Pi.
- Corre de forma nativa en Linux (mavlink-router está pensado precisamente para dispositivos de este tipo).
- Igual que el Bridge, no traduce ni interpreta nada: es un repetidor puro, así que no introduce ningún comportamiento nuevo sobre el protocolo MAVLink.
- Se configura una sola vez con un fichero de texto (los endpoints de entrada y salida), sin depender de la interfaz gráfica de ningún programa concreto.

Para este proyecto, con dos procesos Python independientes (receptor.py y vuelo.py) que necesitan un comportamiento predecible y estable del enlace, sin necesitar en ningún momento las funciones de GCS de mavproxy, mavlink-router es la opción más adecuada — y es la que se usa de forma consistente en la Fase 5 y en el Apéndice B.

Nota: el detalle de cómo se configuran estos reenvíos a nivel de sockets (modo cliente/servidor, bind, puertos) se explica en MAVLink_Explicado.docx, en la sección sobre cómo tener varios programas escuchando a la vez en la Raspberry Pi.

B. Arquitectura en Cloud

Además de los ajustes que tenemos que hacer en la arquitectura local, para tener comunicación con el dron, es importante la arquitectura en Cloud, para que el dron comunique los datos, algo que empezamos a construir en esta fase, con los requerimientos mínimos para empezar, y que iremos potenciando, evolucionando y desarrollado en paralelo durante el resto de fases.

En este proyecto el dron comunica sus datos de vuelo, sensores, etc, a un sistema de control y dashboards en cloud (AWS). En estas fases iniciales, podemos probar también esa conexión y iteración, sin necesidad aun de desplegar los scripts finales en la raspberry (la comunicación puede probarse con nuestra propio portátil, de forma que podemos usar el mismo portátil para las pruebas iniciales, Mission Planner, conexión con cloud, etc). Durante este fase, instalaremos el bróker de mosquito necesario en la nube para el envío de los datos, y los scripts en la nube necesarios para recoger esos datos y almacenarlos en la base de datos InfluxDB, que luego nos permitirá consultarlos o interactuar con ellos a través de Grafana o similares.

Para ello, se habilita la infraestructura base sobre una instancia AWS EC2 dentro de una VPC privada, compuesta por:

- Broker MQTT (Eclipse Mosquitto): Actúa como nodo central de mensajería ligera para recibir tanto los flujos de telemetría como los comandos de misión.
- Base de datos de series temporales (InfluxDB): Almacén optimizado para la persistencia de métricas con alta frecuencia de muestreo y marcas temporales continuas.
- Servicio de ingesta (mqtt_to_influx.py): Demonio puente encargado de suscribirse a los topics del broker y persistir los paquetes JSON estructurados directamente en el bucket de series temporales.
- Red Privada Segura (Tailscale / WireGuard): Creación de una malla privada (VPN) que interconecta de forma segura y transparente el computador de a bordo, el servidor cloud y los puestos de operador, evitando exponer puertos críticos directamente a internet pública.
- Gestión de Dominio y Seguridad Perimetral (Cloudflare): Administración de los registros DNS del proyecto, enrutamiento del tráfico hacia la IP pública de la instancia EC2 y provisión de capas de cifrado SSL/TLS, protección contra ataques y optimización de entrega.

Esta estructura mínima inicial sirve como base extensible sobre la cual se incorporan mejoras incrementales a lo largo del ciclo de vida del proyecto:

- Integración analítica y visual: Conexión de herramientas de visualización (Grafana o paneles web basados en Nginx) para la monitorización gráfica de las misiones SAR.
- Canal bidireccional: Evolución desde un modelo inicial de solo ingesta (subida de telemetría) hacia un esquema con canales dedicados de control remoto y cambio dinámico de modos de vuelo (armado, bajada de órdenes, envío de misiones, etc).
- Mecanismos de tolerancia a fallos: Compatibilidad con la arquitectura store-and-forward (buffer local SQLite) implementada en el companion computer para sincronizar ráfagas de datos acumuladas tras pérdidas temporales de cobertura 4G/LTE.

6. Fase 3 — Control del dron desde la Raspberry Pi

Una vez que ya hemos hecho las primeras pruebas básicas en un entorno simulado de manejar un dron, armarlo, volarlo, introducir misiones, y conocer el manejo y funcionalidad habitual del mismo a través del simulador que nos trae Mission planer, y tras añadir en la segunda fase los elementos necesarios para interactuar directamente con MavLink a través de un MavLink Router/Proxy, ya podemos empezar a desarrollar un entorno de simulación mucho mas real y cercano a la realidad final, donde podemos usar la estructura de scripts en Python que utilizaremos en el dron para interactuar con el, y esta vez corriendo sobre la Raspberry PI que actua como “Edge” o “Computer” companion, no sobre nuestro Pc.

Esta es, con diferencia, la fase más crítica y estimulante del proyecto: aquí es donde empieza de verdad el manejo del dron a través de la Raspberry Pi, sentando las bases de todo el control remoto que se completará en la Fase 6. Se aborda en tres pasos, cada uno añadiendo una sola pieza nueva sobre el anterior, exactamente con la misma filosofía incremental del resto del documento.

6.1 Paso 1 — Un script de Python con argumentos, ejecutado a mano

El primer paso traslada a la Raspberry Pi 5 el mismo papel que tenía el script de Python en la Fase 2, pero reescrito como una herramienta de línea de comandos con varias opciones —armar, despegar, volver a home, ejecutar una misión— que se pasan por parámetro. Es un cambio deliberadamente pequeño: la Raspberry Pi se conecta al MAVLink Bridge exactamente igual que en la Fase 2 (mismo puerto UDP 14551, ahora por Wi-Fi en vez de la conexión local 127.0.0.1 o loopback).

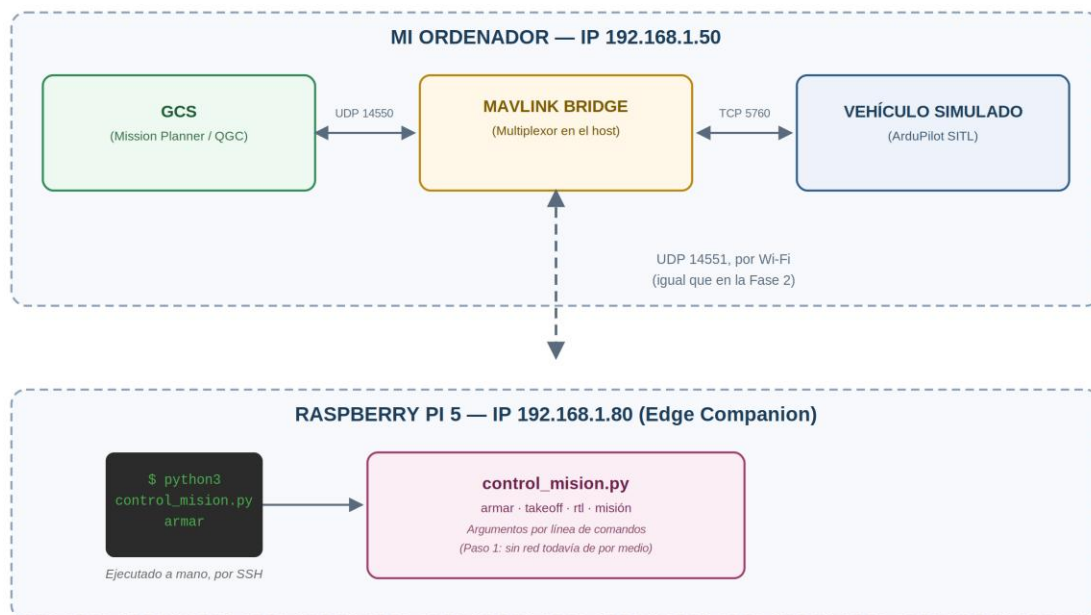


Figura 6. Paso 1: un script de línea de comandos en la Raspberry Pi, ejecutado a mano, envía órdenes al Bridge exactamente igual que en la Fase 2.

- `armar / desarmar`: activa o desactiva los motores.
- `takeoff --altura N`: despegar y sube hasta N metros.

- rtl: vuelve al punto de despegue (Return To Launch).
- mision [--archivo ruta]: sube (si se indica un archivo) y ejecuta una misión, cambiando el vehículo a modo AUTO.

6.2 Paso 2 — El comando se lanza desde la nube, por IoT

El segundo paso traslada el origen del comando desde la propia Raspberry Pi hasta la máquina EC2 de la Infraestructura, reutilizando el mismo broker Mosquitto que ya está desplegado. Un script en el EC2 publica el comando (por ejemplo, "armar") en un topic MQTT dedicado a comandos; en la Raspberry Pi, un segundo script queda suscrito a ese topic de forma continua y, en cuanto recibe un mensaje, invoca exactamente la misma lógica del Paso 1 para trasladarlo al dron por MAVLink.

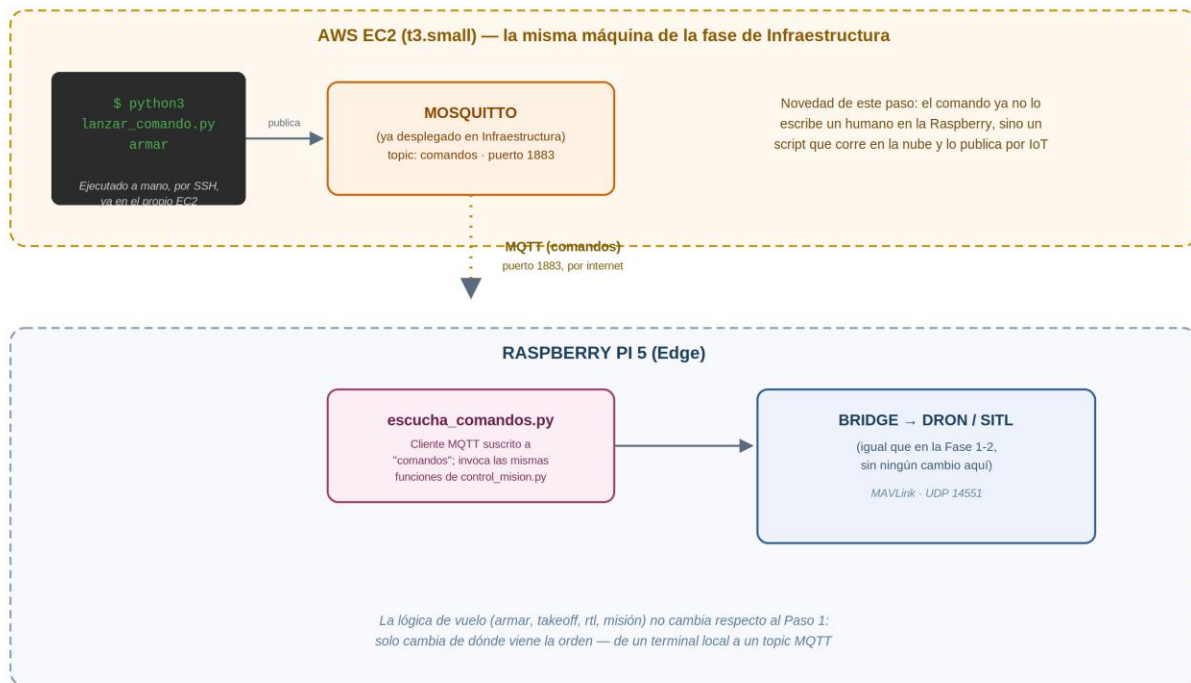


Figura 7. Paso 2: el comando ya no se teclea en la Raspberry Pi, sino que se publica desde el EC2 y viaja por MQTT hasta ella.

Es importante notar lo que NO cambia: la lógica de vuelo (armar, takeoff, rtl, misión) es literalmente la misma que en el Paso 1. Lo único que cambia es de dónde viene la orden — de un terminal local a un mensaje MQTT llegado por internet. Este es exactamente el mismo principio de diseño que ya se aplicó al pasar de la Fase 1 a la Fase 2: separar "qué se hace" de "quién lo pide".

6.3 Paso 3 — Un panel web sustituye al script manual

El tercer paso reemplaza el script que, en el Paso 2, se lanzaba a mano en el EC2, por un servidor web (apoyado en el nginx ya desplegado en la Infraestructura, más un backend sencillo) que publica el mismo mensaje MQTT cuando el operador pulsa un botón desde el navegador. Para la Raspberry Pi no cambia absolutamente nada respecto al Paso 2: sigue suscrita al mismo topic, sin ser consciente de si el mensaje lo publicó un script de consola o un clic en una página web.

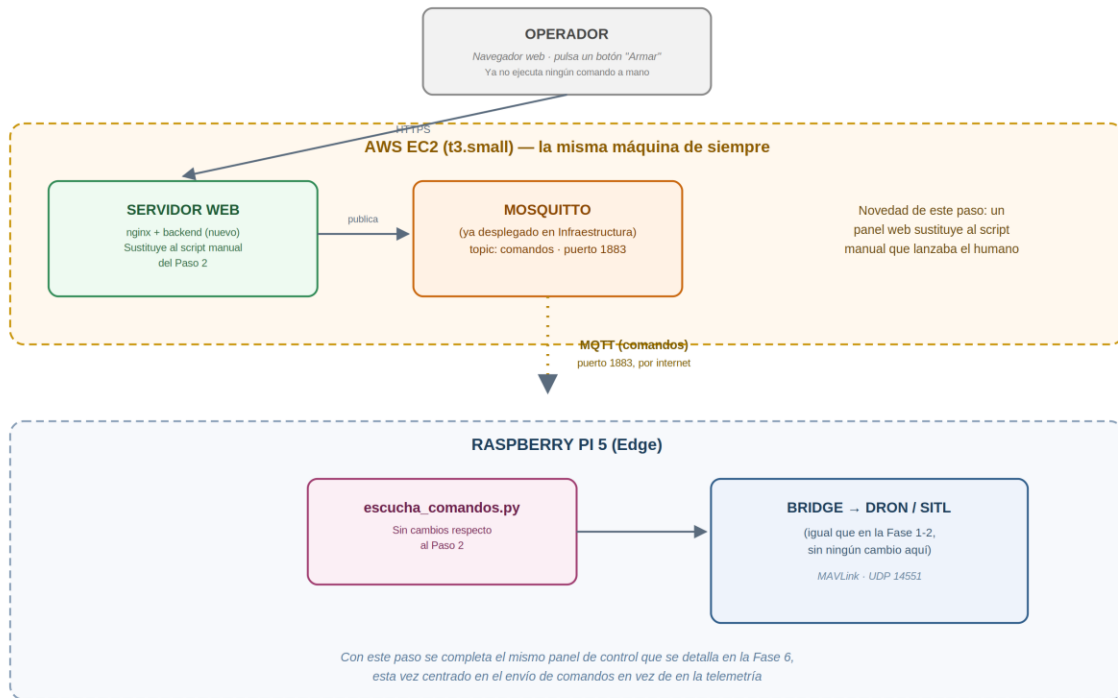


Figura 8. Paso 3: un panel web en el EC2 sustituye al script manual; la Raspberry Pi no nota ninguna diferencia respecto al Paso 2.

Con este paso queda ya construida la mitad "de bajada" (comandos) del panel de control que se completa en la Fase 6, cuando se añada también la mitad "de subida" (telemetría y sensores) para tener visibilidad, no solo control, desde el mismo panel.

7. Fase 4 — HITL

Entre el control remoto (Fase 3) y el vuelo real (Fase 5) existe, en teoría, un paso intermedio muy citado en la bibliografía: HITL, de Hardware In The Loop. La idea es tentadora — la Pixhawk deja de ser un programa (SITL) y pasa a ser la placa física real, mientras un ordenador le sigue inyectando por USB lecturas simuladas de GPS, acelerómetro y barómetro (calculadas por el mismo motor de física que el SITL), de modo que el firmware real reacciona con salidas PWM reales hacia los motores sin que el dron llegue a volar.

Esta fase, sin embargo hemos considerado que no aporta ningún valor al proyecto, y ha sido descartada conscientemente, después de valorar si merecía la pena o no realizarla.

Por qué esta fase no se ejecuta en este proyecto

Antes de intentarlo se investigó cómo funciona en la práctica el HITL de ArduPilot, y el resultado desaconseja perseguirlo aquí:

- 1) Requiere flashear en la Pixhawk un firmware "HIL" específico y separado del firmware de vuelo normal — para Copter, el que documenta oficialmente ArduPilot es una rama muy antigua (anterior a 2014), lo que contradice la propia idea de "probar exactamente el firmware con el que se va a volar".
- 2) El soporte de HITL para Copter aparece, en la documentación y en los foros oficiales de ArduPilot, notablemente menos maduro y menos mantenido que para Plane (ala fija) — varios hilos de usuarios entre 2017 y 2020 documentan configuraciones frágiles (ficheros de protocolo que copiar a mano, versiones concretas de FlightGear/X-Plane) sin garantía de que sigan funcionando en firmware moderno.
- 3) Depende de un simulador de vuelo externo (FlightGear o X-Plane) que hay que instalar y calibrar aparte, y que QGroundControl ni siquiera soporta — solo Mission Planner, y solo con ese firmware antiguo.

Dado el coste de meterse en una rama de firmware obsoleta con herramientas frágiles, y dado que este proyecto ya alcanzó una validación más completa por otra vía (la Fase 5, con hardware real y firmware de producción), no se consideró que mereciera la pena.

Lo que sí se hace en su lugar: un ensayo en tierra sin inyección de sensores

Aunque no se llega a inyectar GPS ni el resto de sensores en la Pixhawk (lo que exigiría el HITL completo descrito arriba), sí se hizo, ya con el hardware definitivo, un ensayo en tierra que cubre buena parte del mismo objetivo — comprobar el conjunto sin arriesgarse a volar:

- Pixhawk real, firmware de producción (el mismo que se usa en vuelo, sin ninguna rama especial), con las hélices desmontadas por seguridad.
- Ciclo completo de armado y desarmado validado por MAVLink real contra ese hardware (documentado en detalle en el Apéndice C, con el script test-arm-serial.py): comando ARM, confirmación del autopiloto, espera, comando DISARM y confirmación — sin llegar a volar en ningún momento.
- En paralelo, para ensayar el resto de la cadena sin depender de que el dron esté realmente en el aire, vuelo.py se configuran los diferentes programas y procesos con un flag --fake, que genera telemetría

de vuelo simulada (posición, altitud, batería) centrada en el campo de pruebas, en vez de leerla de la Pixhawk real.

- Probablemente el script mas importante sea el del propósito del dron, el reconocimiento del video y la localización en el de personas. Para eso necesitaríamos un vuelo real, pero aquí, lo simulamos con videos pregrabados. Para ello, `deteccion.py`, en lugar de la cámara en vivo, se modifica para procesar un vídeo ya grabado de un vuelo real (`samples/vuelo1.mp4`) como si fuera la imagen que llegaría de la cámara en ese instante — permitiendo probar toda la cadena de detección de personas y generación de alertas sin necesidad de que el dron esté volando de verdad en ese momento.

Combinando estas tres piezas se puede ensayar en el banco casi todo el sistema en marcha a la vez — armado real, telemetría fluyendo (aunque simulada), detección y alertas funcionando— sin que el dron llegue a despegar. No es un HITL en el sentido estricto del término (no hay inyección de sensores en el propio autopiloto, y el firmware que corre es el de producción, no uno especial), **pero cubre una parte razonable del mismo objetivo práctico: dar confianza en que el sistema completo funciona junto antes de arriesgarse a un vuelo real.**

8. Fase 5 — Despliegue en hardware real

En esta fase el simulador y la simulación de sensores desaparecen por completo: el dron vuela de verdad, cerrando el círculo con el punto de partida del documento. La Pixhawk real tiene varios puertos serie (UART) independientes, así que ya no hace falta ningún Bridge por software en el sentido de la Fase 2 — pero, como se explica más abajo, sí hace falta su equivalente para hardware real (un MAVLink Proxy) en cuanto hay más de un programa en la Raspberry Pi hablando con la Pixhawk a la vez, que es exactamente el caso de este proyecto.

- Enlace de control (Edge): la Raspberry Pi 5 se conecta por cable, directamente por UART, al puerto TELEM 3 de la Pixhawk, y controla el dron mediante el puerto serie /dev/ttyAMA0 (ver nota de esta misma sección: en esta Raspberry Pi 5 concreta, no es /dev/serial0).
- Enlace de supervisión (GCS): la estación de tierra —Mission Planner, QGC o la que se prefiera— recibe la telemetría mediante el mismo radioenlace de telemetría (433/915 MHz) presentado en el punto de partida, conectado al puerto TELEM 1 de la Pixhawk.
- El puerto TELEM 2 se deja libre, sin usar por ahora: es el puerto natural para añadir el día de mañana otro consumidor MAVLink (por ejemplo, un segundo enlace de radio, un módulo de telemetría de largo alcance, u otro companion computer) sin tener que reconfigurar nada de lo ya montado.

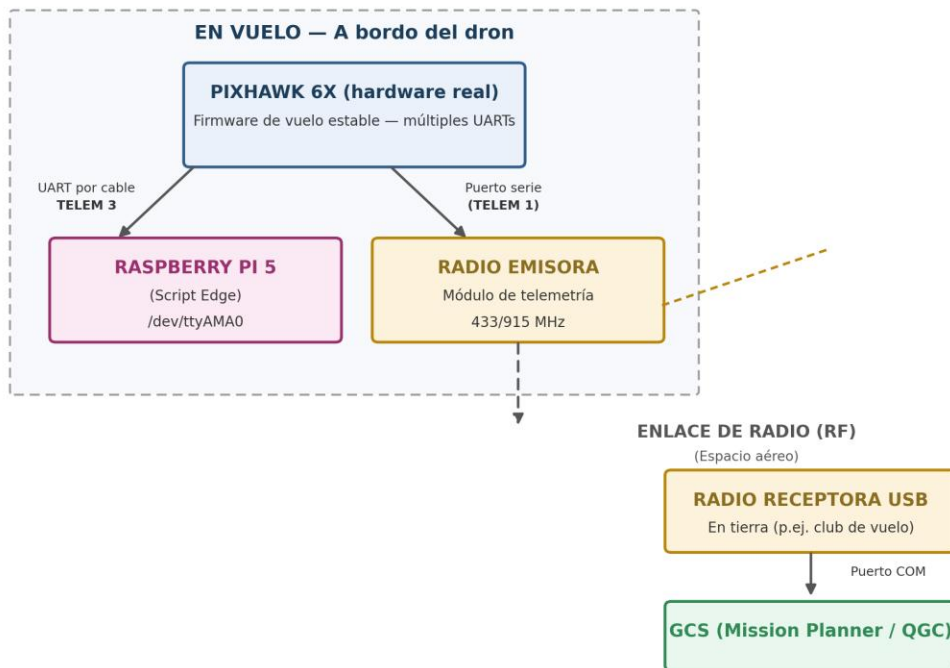


Figura 9). Hardware real embarcado: la Raspberry Pi controla por cable a través de TELEM3 (/dev/ttyAMA0), y la telemetría viaja por el radioenlace TELEM1 hacia la GCS.

8.1 Por qué aquí ya hace falta un MAVLink Proxy, no solo un cable

La Fase 2 dejó planteada la necesidad de un MAVLink Proxy en hardware real como una posibilidad para "algún día", si la Raspberry llegaba a necesitar repartir la telemetría del Pixhawk entre más de un programa local a la vez. En este proyecto ese día es este: la Raspberry Pi no ejecuta un único script de control, sino dos procesos independientes y simultáneos —

- receptor.py: se suscribe a los comandos que llegan por MQTT (armar, desarmar...) y los traduce a MAVLink hacia la Pixhawk.
- vuelo.py: lee de forma continua la telemetría MAVLink de la Pixhawk (posición, batería, modo) y la publica hacia la nube.

Recordatorio de terminología (Fase 2, sección "mavlink-router frente a mavproxy"): "MAVLink Proxy" es el papel genérico — cualquier proceso que concentra un enlace exclusivo y lo reparte entre varios consumidores, sea Mission Planner haciendo de Bridge en SITL, o una herramienta dedicada en hardware real. mavlink-router es la herramienta concreta elegida aquí para desempeñar ese papel, en vez de su alternativa mavproxy, por ser más ligera y no interferir con las frecuencias de telemetría que cada proceso solicita por su cuenta.

Un puerto serie es un recurso exclusivo del sistema operativo: dos procesos no pueden abrir /dev/ttyAMA0 a la vez sin pisarse los datos que entran y salen por TELEM3. La solución es exactamente la misma que ya resolvía este problema en la Fase 2, aplicada ahora al hardware real: un MAVLink Proxy — en este caso, mavlink-router — que sea el único que abre el puerto serie, y que reparte ese tráfico hacia un endpoint de red local por proceso.

De la Fase 2 (SITL + Bridge) a la Fase 5 real (TELEM3 + mavlink-router)

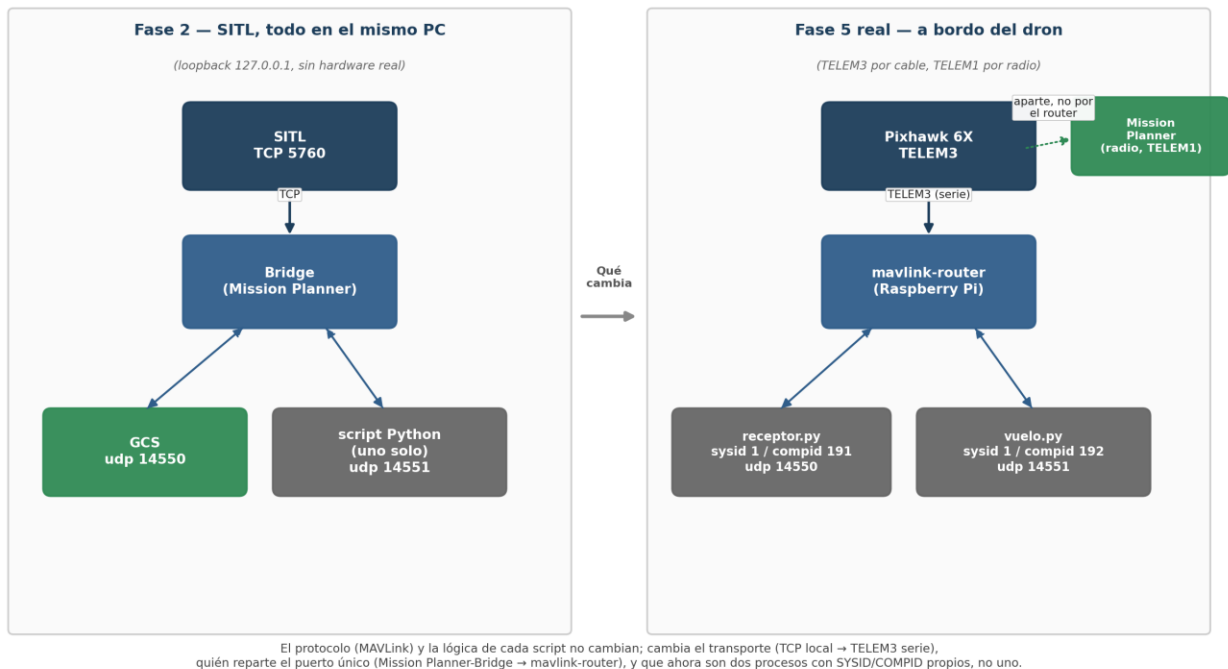


Figura 10. Qué cambia entre la Fase 2 (Bridge/Mission Planner sobre TCP local) y esta Fase 5 real (mavlink-router sobre TELEM3): el protocolo y la lógica de cada script no cambian, cambia el transporte y quién reparte el puerto único.

El Apéndice B, al final de este documento, recoge la instalación, la configuración y las pruebas de mavlink-router para este caso concreto, incluyendo la identificación SYSID/COMPID que conviene dar a cada proceso — un aspecto que la Fase 2 no necesitaba tratar (allí solo había, como mucho, un script y una GCS, cada uno con su puerto UDP claramente separado) y que en la Fase 5 sí importa, al convivir tres orígenes distintos de mensajes MAVLink sobre el mismo vehículo: la GCS, receptor.py y vuelo.py.

9. Fase 6 — Arquitectura cloud: telemetría, sensores y control remoto

Esta última fase añade la mitad que faltaba del panel de control: la telemetría y los sensores subiendo hacia la nube, para completar el canal de comandos que ya se construyó en la Fase 3. El resultado es un panel único, en el mismo servidor web, con visibilidad completa en ambos sentidos. Esta fase tampoco sustituye el esquema de la Fase 5: lo completa. El radioenlace de telemetría hacia la GCS de campo se mantiene exactamente igual, y en paralelo se añade una segunda vía, a través de la nube.

Dos ordenadores, dos formas de supervisar el mismo dron

Conviene distinguir dos equipos con papeles distintos. El Ordenador 1 está físicamente en el campo de vuelo, junto al piloto, y recibe la telemetría en directo por el mismo radioenlace de la Fase 5 (Mission Planner o QGC, sin depender de que haya internet). El Ordenador 2 es el del operador remoto: no necesita estar cerca del dron, y accede desde un navegador web al panel de control alojado en el servidor de la nube —el mismo nginx de la Infraestructura y de la Fase 3—, viajando la información por internet en lugar de por radiofrecuencia.

En el lado del dron (Edge)

El programa Python que corre en la Raspberry Pi 5 pasa a tener dos fuentes de datos: la telemetría MAVLink que recibe de la Pixhawk por el puerto serie (igual que en la Fase 5), y las lecturas de un sensor ambiental BME680 (temperatura, humedad y presión) conectado directamente a la Raspberry Pi por I2C — el mismo sensor que ya se leía en la Infraestructura, ahora combinado con datos de vuelo. Cada lectura se guarda primero en una base de datos local (SQLite), que actúa como memoria intermedia: si la conexión a internet falla, los datos no se pierden, se siguen acumulando en la Raspberry Pi y se envían más tarde en cuanto la red vuelve (arquitectura store-and-forward, o "edge-first").

Nota: como se detalla en la Fase 5 y en el Apéndice B, en este proyecto "el programa que lee la telemetría" es concretamente vuelo.py, uno de los dos procesos que comparten el puerto serie de la Pixhawk a través de mavlink-router — no un único script monolítico.

En el lado de la nube (AWS EC2)

El mismo broker Mosquitto de la Infraestructura recibe por MQTT, en el puerto 1883, todo lo que la Raspberry Pi va publicando. Un pequeño script puente se suscribe a ese broker y guarda cada lectura en la misma InfluxDB, ya desplegada desde el principio. El mismo servidor web que en la Fase 3 ya publicaba comandos, ahora también lee de InfluxDB para mostrar la telemetría y las lecturas ambientales en tiempo real: el panel de control queda así completo, en ambos sentidos, sobre la infraestructura que se montó una sola vez.

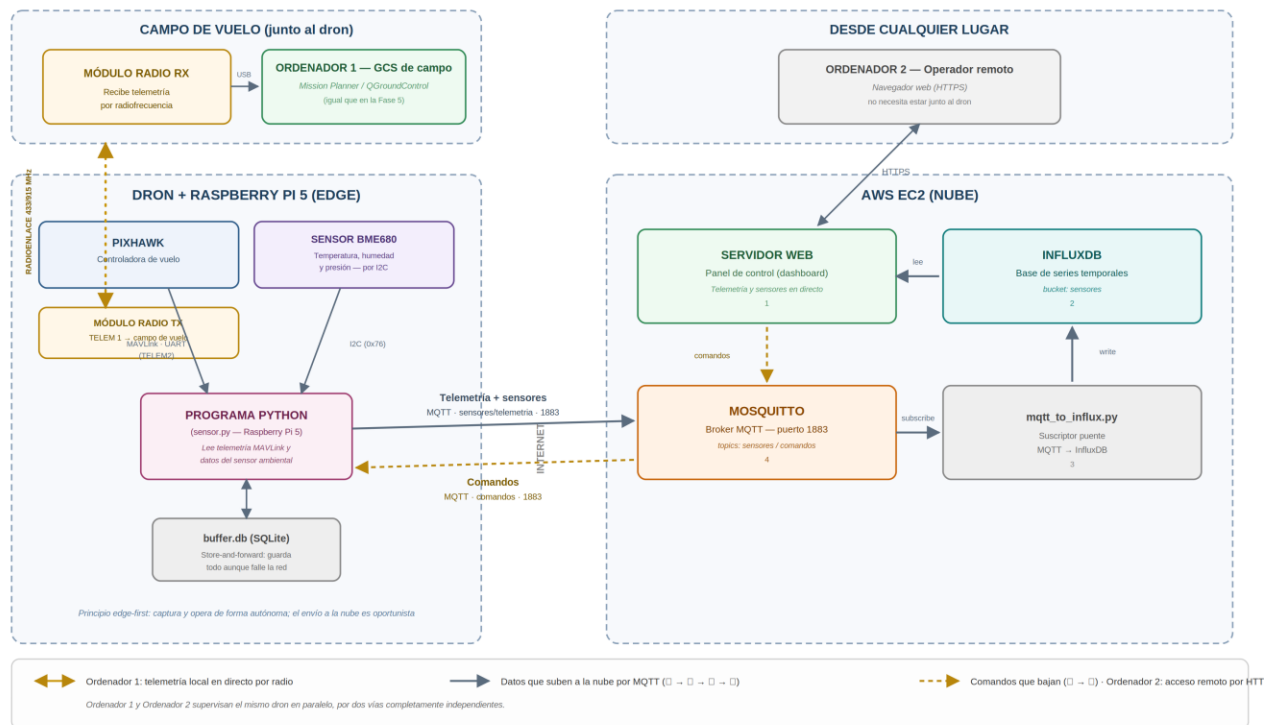


Figura 11. Arquitectura completa: telemetría por radio hacia el campo de vuelo (Ordenador 1) y, en paralelo, telemetría y sensores por MQTT hacia la nube, supervisada desde cualquier lugar (Ordenador 2).

El canal de comandos, ya construido en la Fase 3

La comunicación con la nube no es solo de subida. El broker MQTT ya permitía, desde la Fase 3, el camino inverso: el operador, desde el panel de control web, publica comandos en un topic distinto (por ejemplo, para modificar una ruta o forzar un aterrizaje), que la Raspberry Pi recibe suscrita a ese topic y traslada al dron por MAVLink. Esta fase no añade nada nuevo en ese sentido: simplemente reutiliza, para la telemetría, el mismo patrón que ya funcionaba para los comandos.

Puertos y protocolos de esta fase

- Radiofrecuencia (433/915 MHz): telemetría hacia el Ordenador 1, en el campo de vuelo — igual que en la Fase 5, sin cambios.
- MQTT, puerto 1883: protocolo ligero de publicación/suscripción, pensado para redes inestables; lo usan tanto los datos que suben a la nube como los comandos que bajan.
- I2C (dirección 0x76): conexión del sensor BME680 a la Raspberry Pi.
- MAVLink por UART (/dev/ttyAMA0, vía mavlink-router): igual que en la Fase 5, para leer la telemetría de la Pixhawk.
- HTTPS: acceso del Ordenador 2 al panel de control desde el navegador, desde cualquier lugar.

10. Resumen de todas las etapas

La siguiente tabla recoge, de un vistazo, qué se añade en cada etapa y qué problema resuelve o qué objetivo valida.

Etapas	¿Qué hay de nuevo?	¿Qué problema resuelve o qué valida?
Estudio del Arte	Normativa, permisos, vuelo real, investigación cloud/IoT	Sienta las bases: qué se quiere reproducir y con qué tecnologías
Infraestructura	EC2 t3.small + Mosquitto + InfluxDB + nginx + Raspberry	Valida el canal edge-nube con un caso simple, antes de meter el dron
Fase 1	GCS conectada directo al SITL	Valida que el simulador se comporta como una Pixhawk real (TCP 1 a 1)
Fase 2	MAVLink Bridge + programa Python	Permite que script y GCS controlen el mismo dron simulado a la vez (UDP)
Fase 3	Control desde la Raspberry: CLI → IoT → panel web	Traslada el control del dron a la nube, sin tocar la lógica de vuelo
Fase 4	HITL evaluado y descartado con criterio; ensayo en tierra con --fake de vuelo.py/vídeo pregrabado	Documenta por qué el HITL completo no aporta valor aquí, y qué validación en tierra lo sustituye
Fase 5	Pixhawk física (TELEM3) + mavlink-router + radioenlace real (TELEM1)	Confirma que el software funciona sin cambios sobre hardware embarcado, en vuelo, con dos procesos compartiendo el puerto serie
Fase 6	Se añade telemetría + sensores al mismo panel	Completa, con la subida de datos, el panel construido en la Fase 3

11. Conclusión metodológica

Este recorrido en ocho etapas aporta un valor metodológico claro para la memoria del proyecto: empieza por entender bien el problema y montar una infraestructura mínima y validada, reproduce el sistema real en un entorno de simulación cada vez más completo, traslada el control a la nube, pasa por hardware real todavía en tierra, y termina devolviendo ese mismo software al mundo real, ahora con supervisión y control remotos completos.

- Estudio del Arte: entender el sistema real, la normativa y las tecnologías que se van a usar.
- Infraestructura: montar y validar el canal edge-nube antes de complicarlo con el dron.
- Fase 1: validar que el simulador se comporta como una Pixhawk real.
- Fase 2: resolver la limitación del puerto único e incorporar el control automático por Python, añadiendo no solo la Arquitectura local necesaria, sino también realizando la estructura base de la Arquitectura Cloud que usaran nuestros scripts.
- Fase 3: trasladar el control del dron a la Raspberry Pi y, de ahí, a la nube — sin tocar la lógica de vuelo.
- Fase 4: evaluar con criterio si el HITL completo aporta valor en este proyecto (no lo hace, por depender de firmware legacy y soporte frágil para Copter) y documentar el ensayo en tierra que lo sustituye.
- Fase 5: confirmar que el software funciona sin cambios sobre hardware real, en vuelo, resolviendo de nuevo la limitación del puerto único — ahora con mavlink-router en vez del Bridge de la Fase 2.
- Fase 6: añadir la telemetría y los sensores al mismo panel de control construido en la Fase 3.

En ningún momento del recorrido cambia la lógica del programa de control ni el protocolo MAVLink: solo cambian, etapa a etapa, la ubicación física de cada componente y el medio de transporte de los datos (loopback, Wi-Fi local, radioenlace, MQTT o internet). Ese es precisamente el argumento que sostiene la validez de todo el trabajo de simulación de cara al vuelo real.

Apéndice A — Equivalencia con una solución propietaria: el DJI Neo 2

Todo lo descrito en este documento es un enfoque abierto, basado en ArduPilot, MAVLink y componentes que se integran a mano: Pixhawk, radios de telemetría, Raspberry Pi, Bridge, broker MQTT propio, etc. Resulta útil comparar esta arquitectura con un dron comercial cerrado, como el DJI Neo 2 (lanzado en noviembre de 2025), para ver qué roles cumple —y con qué piezas— un fabricante que integra todo de fábrica.

Diferencia de fondo: piezas separadas frente a un sistema cerrado

El DJI Neo 2 no tiene Pixhawk, no habla MAVLink y no expone ningún puerto TELEM al usuario. DJI sustituye toda la arquitectura abierta que hemos ido montando por un único enlace digital propietario que transporta a la vez vídeo, telemetría y comandos de control, cifrado y cerrado, sin documentación pública. Es, en cierto modo, el mismo objetivo final (pilotar un dron y ver sus datos) resuelto con filosofía inversa: en vez de piezas intercambiables que uno mismo conecta, todo viene integrado y no se puede sustituir ni inspeccionar.

Rol en nuestro sistema	Piezas que usamos (abierto)	Equivalente en el DJI Neo 2 (cerrado)
Controladora de vuelo	Pixhawk + firmware ArduPilot (abierto)	Controladora de vuelo propietaria de DJI (cerrada, no documentada)
Protocolo de control	MAVLink (abierto, estándar)	Protocolo propietario de DJI, cifrado
Estación de tierra (GCS)	Mission Planner / QGroundControl	App DJI Fly, en el móvil del piloto
Enlace de control manual	Receptor + mando RC, pareja propia (p.ej. 2,4 GHz)	Wi-Fi integrado al móvil (~100 m) o, con el Transceiver opcional, enlace O4 (hasta 6-10 km) con mando RC-N3 o Goggles N3
Enlace de telemetría	Par de radios TX/RX dedicado (433/915 MHz), añadido aparte	El mismo enlace integrado de arriba: vídeo, control y telemetría viajan juntos
Procesamiento a bordo / control automático	Raspberry Pi 5 + programa Python (programable por el usuario)	Chip de a bordo de DJI con ActiveTrack, SelfieShot, vuelo por gestos (cerrado, no programable)
Sensores adicionales	BME680 (temperatura, humedad, presión) por I2C	Sensores de visión, LiDAR frontal e infrarrojos, solo para evitar obstáculos
Supervisión y control en la nube	AWS EC2 + Mosquitto (MQTT) + InfluxDB + dashboard propio	No disponible en el Neo 2; existe solo en la gama profesional de DJI (Matrice + FlightHub 2)

Lecturas clave de la comparación

- Donde nuestro sistema necesita dos parejas de radio distintas (telemetría + RC) más, opcionalmente, un tercer enlace de vídeo, el DJI Neo 2 básico lo resuelve con un único enlace Wi-Fi integrado hacia el móvil (hasta ~100 m), que lleva vídeo, telemetría y control a la vez.
- Para más alcance (hasta 6-10 km), el Neo 2 necesita añadir un periférico opcional, el DJI Neo 2 Digital Transceiver, junto con un mando RC-N3 o las gafas Goggles N3 — un paralelismo directo con tener que añadir nuestra pareja de radios de telemetría a la Pixhawk.

- El Neo 2 sí tiene el equivalente a nuestro Bridge y nuestro programa Python de control automático (ActiveTrack, SelfieShot, vuelo por gestos...), pero corre en un chip cerrado de DJI: no es programable ni sustituible por el usuario, a diferencia de nuestra Raspberry Pi.
- El Neo 2 no tiene ningún equivalente de fábrica a nuestra Fase 6: no hay un broker MQTT ni una base de datos ni un panel web propios accesibles por el usuario. DJI reserva ese nivel de telemetría en la nube para su gama empresarial (drones Matrice con la plataforma FlightHub 2), fuera del segmento al que pertenece el Neo 2.
- El Neo 2 sí resuelve, de fábrica, el enlace de vídeo que hemos dejado fuera del punto de partida: va integrado en el mismo enlace digital que el control y la telemetría, algo que en un sistema abierto como el nuestro hay que añadir aparte (por ejemplo, streaming por TCP/IP o un enlace de radio dedicado).

En definitiva, el DJI Neo 2 demuestra que los mismos roles de esta guía —controladora de vuelo, GCS, enlace de control, enlace de datos, procesamiento a bordo y supervisión en la nube— existen también en un producto comercial cerrado; solo que, en vez de siete piezas que se combinan y se pueden auditar una a una, están fusionadas en un solo sistema propietario que no se puede abrir ni reprogramar.

Una ventaja práctica: usarlo como referencia ya validada

Más allá de la comparación teórica, disponer de un DJI Neo 2 aporta una ventaja muy concreta para el proyecto: permite probar todo el concepto de misión sobre un ecosistema propietario ya maduro, probado y fiable, y usar lo que graba como referencia frente a lo que luego obtengamos con nuestro propio sistema abierto.

Un flujo de trabajo típico sería: volar una misión real con el Neo 2 (por ejemplo, un recorrido de búsqueda sobre una zona de prueba); al terminar, acceder a su tarjeta de almacenamiento interno y extraer tanto el vídeo grabado como el registro de vuelo (el track GPS real, con su posición, altura y velocidad en cada instante); y, a partir de ese mismo track, programar en el simulador (SITL) una misión que reproduzca exactamente el mismo recorrido. De esa forma, el resultado de nuestro propio sistema puede compararse directamente contra un vuelo real ya conocido, en lugar de contra una misión inventada sin referencia.

Además, el vídeo grabado por el Neo 2 durante ese vuelo real puede reutilizarse dentro de la simulación como fuente de imagen de cámara —por ejemplo, para probar en SITL un algoritmo de procesamiento de vídeo o de detección de objetos sin necesidad de volver a volar—, aprovechando que el recorrido y las imágenes ya corresponden a un vuelo real y no a una escena sintética.

Apéndice B — mavlink-router en la práctica: instalación, configuración y SYSID/COMPID

Este apéndice desarrolla, con el mismo nivel de detalle práctico que el Apéndice A dedica al DJI Neo 2, el MAVLink Proxy que la Fase 5 (sección 8.1) identifica como necesario desde el primer despliegue en hardware real de este proyecto: mavlink-router, instalado en la Raspberry Pi 5, repartiendo TELEM3 entre receptor.py y vuelo.py.

B.1 Prerrequisitos

- UART de TELEM3 habilitado en la Raspberry Pi y accesible como /dev/ttyAMA0 — NO necesariamente /dev/serial0, ver nota en la sección 8 (raspi-config → Interface Options → Serial Port: consola serie desactivada, hardware serial habilitado).
- Parámetros ArduPilot ya configurados en el SERIALx correspondiente a TELEM3 (SERIAL5 en un Pixhawk 6X): SERIALx_PROTOCOL=2 (MAVLink2) y SERIALx_BAUD acorde (57 para 57600 baudios, el valor usado en este proyecto).
- Paquetes de compilación: git, meson (≥0.55), ninja-build, pkg-config, gcc, g++ y systemd.

B.2 Instalación (compilación desde fuente)

Raspberry Pi OS no distribuye mavlink-router como paquete apt estable en todas sus versiones, así que la vía fiable es compilar desde el repositorio oficial. Para eso se usan meson y ninja: meson es la herramienta que lee la configuración del proyecto (qué ficheros hay que compilar, con qué opciones, qué dependencias necesita) y prepara todo lo necesario para construirlo; ninja es el programa que ejecuta esa compilación en sí, de forma rápida. En la práctica, para quien instala mavlink-router basta con saber que meson setup "prepara" la compilación y ninja la "ejecuta":

```
sudo apt update
sudo apt install -y git meson ninja-build pkg-config gcc g++ systemd libsystemd-dev

cd ~
git clone https://github.com/mavlink-router/mavlink-router.git
cd mavlink-router
git submodule update --init --recursive

meson setup build . -Dsystemdsystemunitdir=/usr/lib/systemd/system
ninja -C build
sudo ninja -C build install

mavlink-routerd --version    # confirma que el binario quedó instalado
```

Nota validada en la práctica durante la instalación real de este proyecto

libsystemd-dev es imprescindible: sin él, meson falla con "Dependency \"systemd\" not found". No basta con el paquete systemd (ya instalado en cualquier Raspberry Pi OS); hace falta específicamente la librería de desarrollo.

El flag `-Dsystemdsystemunitdir=/usr/lib/systemd/system` evita un segundo fallo posterior ("Could not get pkg-config variable") que aparece incluso con `libsystemd-dev` ya instalado, en sistemas donde el `.pc` de `systemd` se llama `libsystemd.pc` en vez de `systemd.pc` (`systemd` 257 en Debian trixie, entre otros). Si `meson` sigue sin encontrar la dependencia `systemd` a pesar de tener `libsystemd-dev` instalado, comprobarlo con `pkg-config --modversion systemd`; si falla, buscar el fichero real con `dpkg -L libsystemd-dev | grep pkgconfig` y crear un enlace simbólico a `systemd.pc` apuntando a ese fichero. Cada vez que se repita `meson setup` tras un intento fallido, borrar antes la carpeta `build/` (`rm -rf build`) — `meson` cachea el resultado del intento anterior y puede no detectar que una dependencia recién instalada ya está disponible.

B.3 Fichero de configuración: `/etc/mavlink-router/main.conf`

El fichero no se crea automáticamente en la instalación; hay que crearlo a mano. Un endpoint por proceso, ambos en modo Normal — `mavlink-router` envía activamente a esa dirección:puerto, igual que hace Mission Planner en modo SITL en la Fase 2 (sección 5), así el lado Python no cambia de patrón entre SITL y hardware real:

```
[General]
TcpServerPort = 0
ReportStats = false
MavlinkDialect = ardupilotmega

[UartEndpoint telem3]
Device = /dev/ttyAMA0
Baud = 57600

[UdpEndpoint receptor]
Mode = Normal
Address = 127.0.0.1
Port = 14550

[UdpEndpoint vuelo]
Mode = Normal
Address = 127.0.0.1
Port = 14551
```

- `TcpServerPort=0` desactiva el servidor TCP por defecto (puerto 5760, el mismo número que usa el SITL nativo de la Fase 1 — aquí no hace falta).
- Baud debe coincidir exactamente con el `SERIALx_BAUD` configurado en ArduPilot para TELEM3.
- `Mode = Normal` hace que `mavlink-router` envíe activamente cada mensaje a `127.0.0.1:puerto`; `receptor.py` y `vuelo.py` simplemente escuchan ahí, igual que en la Fase 2 la GCS y el script escuchaban lo que el Bridge de Mission Planner les reenviaba.

B.4 Arrancar como servicio `systemd`

```
sudo systemctl daemon-reload
sudo systemctl enable mavlink-router.service
sudo systemctl start mavlink-router.service
sudo systemctl status mavlink-router.service
```

```
# Logs en vivo si algo falla:
journalctl -u mavlink-router.service -f
```

B.5 Verificación antes de conectar receptor.py y vuelo.py

Con el Pixhawk encendido y TELEM3 cableado, el servicio activo, y antes de tocar los scripts propios, conviene confirmar con una herramienta neutra que el enrutado funciona:

```
python3 -c "
from pymavlink import mavutil
m = mavutil.mavlink_connection('udpin:127.0.0.1:14550')
m.wait_heartbeat()
print('OK, sistema', m.target_system, 'componente', m.target_component)
"
```

- Repetir el mismo comando apuntando al puerto 14551 confirma que el segundo endpoint también recibe heartbeats simultáneamente — la prueba de que el fan-out funciona antes de meter la lógica de receptor.py y vuelo.py.

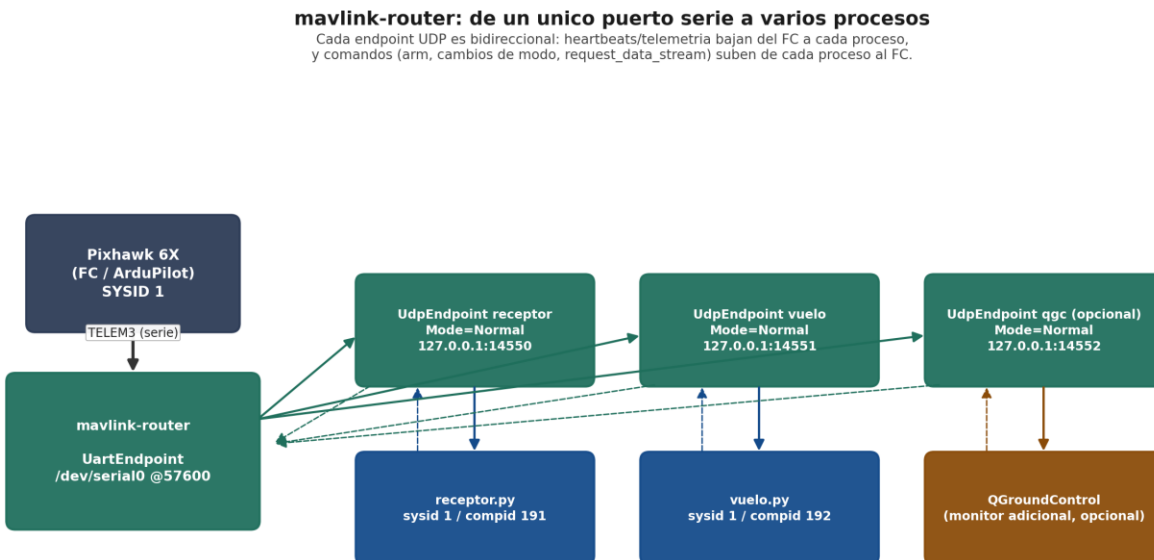


Figura 12. mavlink-router como único propietario del puerto serie TELEM3, repartiendo el tráfico a un endpoint UDP por proceso — el mismo papel que el Bridge de Mission Planner cumplía en la Figura 5 (Fase 2), ahora sobre hardware real.

B.6 receptor.py y vuelo.py: un parámetro para elegir SITL o real

Siguiendo el mismo principio de diseño incremental de todo este documento ("separar qué se hace de quién lo pide", ya aplicado en la Fase 2 y en la Fase 3), receptor.py y vuelo.py se adaptan para elegir su conexión MAVLink mediante una variable de entorno, sin duplicar código ni tocar la lógica de vuelo:

```
MAVLINK_MODE = os.getenv("MAVLINK_MODE", "sitl").strip().lower() # "sitl" | "real"

if MAVLINK_MODE == "real":
    MAVLINK_CONN = os.getenv("MAVLINK_CONN_REAL_RECEPTOR", "udpin:127.0.0.1:14550")
else:
    MAVLINK_CONN = os.getenv("MAVLINK_CONN_SITL_RECEPTOR",
                             os.getenv("MAVLINK_CONN", "udpin:127.0.0.1:14550"))

MAVLINK_SYSID = int(os.getenv("MAVLINK_SYSID", 1))
MAVLINK_COMPID = int(os.getenv("MAVLINK_COMPID_RECEPTOR", 191)) # ver B.7

master = mavutil.mavlink_connection(
    MAVLINK_CONN,
    source_system=MAVLINK_SYSID,
    source_component=MAVLINK_COMPID,
)
```

vuelo.py recibe el mismo tratamiento con MAVLINK_CONN_REAL_VUELO / MAVLINK_CONN_SITL_VUELO y MAVLINK_COMPID_VUELO. El cambio entre SITL y real queda así reducido a una línea del .env, sin tocar código:

```
# Para trabajar contra Mission Planner / SITL (Fases 1-4, por defecto):
MAVLINK_MODE=sitl

# Para trabajar contra la Pixhawk real, vía mavlink-router (Fase 5):
MAVLINK_MODE=real
```

B.7 SYSID/COMPID: por qué hay que añadirlo y cómo

Cada mensaje MAVLink va etiquetado con un SYSID (identifica el sistema/vehículo de origen) y un COMPID (identifica el componente concreto dentro de ese sistema que lo envió). Es lo que permite que el autopiloto, Mission Planner, o cualquier herramienta de inspección MAVLink sepan de quién viene cada mensaje y a quién dirigir cada respuesta — y es la base para poder restringir en el futuro qué componente tiene permiso para hacer qué (por ejemplo, que solo receptor.py pueda armar el vehículo).

Ni receptor.py ni vuelo.py pasaban, antes de este cambio, source_system ni source_component al crear la conexión, así que hay que añadirlo explícitamente. Sin indicarlo, pymavlink aplica sus valores por defecto: source_system=255 y source_component=0 — los mismos que usan por convención las estaciones de tierra, incluida Mission Planner. Mientras no se corrija, esto tiene consecuencias concretas: receptor.py y vuelo.py se identifican ante el autopiloto exactamente igual entre sí, y de forma indistinguible de Mission Planner; y en los logs del autopiloto y en cualquier inspector MAVLink no hay forma de saber qué mensaje vino de cuál de los tres.

Lo que hay que hacer es asignar a cada proceso un SYSID y un COMPID propios y pasarlos explícitamente a mavutil.mavlink_connection() mediante source_system y source_component (ya recogido en el código de B.6). MAVLink reserva un rango de COMPID específico para computadoras de a bordo (companion

computers): MAV_COMP_ID_ONBOARD_COMPUTER (191) a MAV_COMP_ID_ONBOARD_COMPUTER4 (194). La convención habitual es usar el mismo SYSID que el propio vehículo (por defecto 1 en ArduPilot, parámetro SYSID_THISMAV) y diferenciarse solo por COMPID:

Origen	SYSID	COMPID	Rol
Pixhawk 6X (autopiloto)	1	1 AUTOPILOT1	Vehículo — ya asignado por ArduPilot, no se toca.
receptor.py	1	191 ONBOARD_COMPUTER	Comandos: armar, desarmar, cambios de modo.
vuelo.py	1	192 ONBOARD_COMPUTER2	Solo lectura: telemetría de vuelo.
Mission Planner	255	0 GCS	Estación de tierra — sin cambios, valores por defecto de la herramienta.

Posible incidencia a vigilar

Con algunas versiones de ArduPilot, ciertos comandos críticos (en particular cambios de modo y armado) sólo se aceptan si el componente de origen está reconocido como una estación de tierra válida. Hay reportes de usuarios de ArduPilot a los que el compid 191 (MAV_COMP_ID_ONBOARD_COMPUTER) les funcionó para telemetría pero no para comandos de armado/cambio de modo, teniendo que recurrir a compid 190 (MAV_COMP_ID_MISSIONPLANNER) para que el comando fuera aceptado. Esto es dependiente de la versión y configuración del autopiloto, no un comportamiento garantizado del estándar MAVLink. Si en las pruebas de B.8 el comando de armado desde receptor.py no es aceptado a pesar de heartbeats correctos, este es el primer punto a revisar.

B.8 Procedimiento de prueba y validación

Antes de dar por buena la migración a modo real, seguir este orden — cada paso depende de que el anterior haya funcionado, igual que cada fase de este documento depende de la anterior:

- Banco, sin hélices: arrancar mavlink-router y confirmar heartbeats en los puertos 14550 y 14551 (B.5). Con MAVLINK_MODE=real, arrancar primero vuelo.py y confirmar que llegan lat/lon/batería reales del Pixhawk, no ceros; luego receptor.py, confirmando el mensaje de autopiloto conectado.
- Banco, sin hélices, con comando de armado: publicar un "arm" por MQTT contra receptor.py y confirmar en Mission Planner (que sigue viendo el vehículo por TELEM1/radio) que el estado pasa a ARMADO. Si no se acepta, revisar ARMING_CHECK y GPS fix antes de sospechar de SYSID/COMPID, y solo entonces probar compid 190 como se indica en B.7.
- Con hélices, dron sujeto/atado: repetir el armado con hélices montadas, confirmando que los motores responden y que el desarmado de emergencia (por MQTT y por la emisora RC) corta inmediatamente.
- Vuelo corto supervisado: primero en modo manual (STABILIZE/ALT_HOLD) con la emisora en la mano, para validar que TELEM3 y mavlink-router no han introducido ningún problema de vuelo en sí; solo

después, con receptor.py/vuelo.py activos en modo real, monitorizando Mission Planner por TELEM1 en todo momento.

B.9 Checklist final

- [] TELEM3 cableado a UART de la Raspberry Pi; SERIALx_PROTOCOL=2 y SERIALx_BAUD configurados en ArduPilot; TELEM2 queda libre.
- [] UART habilitado en raspi-config, consola serie desactivada.
- [] mavlink-router compilado, instalado y con main.conf apuntando a /dev/ttyAMA0 (NO /dev/serial0 en esta placa) y a los puertos 14550/14551.
- [] Servicio mavlink-router.service activo y habilitado en el arranque.
- [] Prueba neutra de heartbeat en ambos puertos UDP (B.5) superada.
- [] receptor.py y vuelo.py actualizados con MAVLINK_MODE y sysid/compid propios (191/192).
- [] Pruebas de banco sin hélices, con hélices atado, y primer vuelo manual completadas en ese orden antes de operar con receptor.py/vuelo.py en vuelo real.
- [] test-serial.py / test-mavlink.py / test-arm-serial.py / test-estado.py guardados en la Raspberry Pi para futuras comprobaciones (Apéndice C).

Apéndice C — Scripts de prueba de la conexión serie TELEM3

Cuatro scripts pequeños e independientes, sin MQTT ni dominios, contruidos durante la validación real de este proyecto para aislar el enlace TELEM3 paso a paso. Fueron precisamente los que permitieron descubrir que `/dev/serial0` no era el dispositivo correcto en esta Raspberry Pi 5 (ver nota de la sección 8): `test-serial.py` confirmó primero que el UART de la Raspberry Pi funcionaba, y `test-mavlink.py` confirmó después que, apuntando al dispositivo correcto (`/dev/ttyAMA0`), la Pixhawk respondía de verdad.

Script	Para qué sirve
<code>test-serial.py</code>	Confirma que el UART de la Raspberry Pi funciona, sin ningún cable a la Pixhawk (loopback TX-RX en los propios pines 8 y 10).
<code>test-mavlink.py</code>	Confirma que llega un heartbeat real de la Pixhawk por TELEM3 — la primera prueba con el autopiloto de verdad conectado.
<code>test-arm-serial.py</code>	Prueba el ciclo completo de armado y desarmado (con confirmación de seguridad y opción de forzar el armado), por serie directo o a través de mavlink-router.
<code>test-estado.py</code>	Panel en vivo de batería, GPS y RC — incluye los mensajes reales de pre-arm del autopiloto — por serie directo o a través de mavlink-router.

C.1 `test-serial.py` — loopback en la propia Raspberry Pi

Es la prueba más básica, y la que resolvió el caso real de este proyecto: comprueba que el UART de la Raspberry Pi funciona, sin ningún cable a la Pixhawk — se puentea directamente TX con RX en los propios pines 8 y 10 del header GPIO, y el script envía un mensaje de texto por TX esperando recibirlo de vuelta por RX. Si esto falla, el problema está en la propia Raspberry Pi (UART mal habilitado, dispositivo equivocado, o el puente físico mal puesto), no en el cable ni en la Pixhawk. Requiere el paquete `pyserial` (`pip install pyserial` dentro del entorno virtual del proyecto).

```
import serial
import time

DEVICE = '/dev/ttyAMA0'
BAUD = 57600
MENSAJE = b'TEST_LOOPBACK_OK\n'

print(f"Abriendo {DEVICE} a {BAUD} baudios...")
s = serial.Serial(DEVICE, BAUD, timeout=2)
s.reset_input_buffer()

print(f"Enviando: {MENSAJE}")
s.write(MENSAJE)
time.sleep(0.2)

respuesta = s.readline()
print(f"Recibido: {respuesta}")

if respuesta == MENSAJE:
    print("\n--> [OK] El puente TX-RX funciona correctamente.\n")
else:
    print(f"\n--> [FALLO] No ha llegado el mismo mensaje. Recibido: {respuesta!r}\n")
```

```
s.close()
```

C.2 test-mavlink.py — heartbeat real contra la Pixhawk

El mismo tipo de prueba que test-serial.py, pero ya con el cable de la Pixhawk conectado (no el jumper de loopback) y hablando MAVLink de verdad en vez de un mensaje de texto plano. Es la primera confirmación de que hay un autopiloto real respondiendo al otro lado del cable. Antes de ejecutarlo, hay que parar mavlink-router si está corriendo, porque este script abre el puerto serie directamente y solo lo puede tener abierto un proceso a la vez.

```
from pymavlink import mavutil

DEVICE = '/dev/ttyAMA0'
BAUD = 57600

print(f"Conectando a {DEVICE} a {BAUD} baudios...")
m = mavutil.mavlink_connection(DEVICE, baud=BAUD)

print("Esperando heartbeat (puede tardar unos segundos)...")
m.wait_heartbeat()

print(f"\n--> OK: heartbeat recibido")
print(f"    Sistema: {m.target_system}")
print(f"    Componente: {m.target_component}")
```

C.3 test-arm-serial.py — ciclo completo de armado y desarmado

Va un paso más allá de test-mavlink.py: no solo confirma que hay conexión, sino que prueba el ciclo real de armar y desarmar el vehículo, con todo el detalle que hace falta para depurar un rechazo de armado sin tener que levantar receptor.py.

- Pide una confirmación explícita (escribir CONFIRMO) antes de tocar los motores — pensado para banco de pruebas con las hélices desmontadas, o con el dron sujeto/atado.
- Envía el ARM normal (respetando los pre-arm checks) y captura los STATUSTEXT reales del autopiloto si lo rechaza (p. ej. "PreArm: GPS: no fix", "Arm: Roll (RC1) is not neutral" — este último, un caso real encontrado durante la validación de este proyecto por un desajuste de trim en la calibración de radio), en vez de quedarse solo con un "no se ha podido armar" genérico.
- Si el ARM normal falla, pregunta explícitamente si se quiere FORZAR el armado saltándose esos chequeos (equivalente al botón "Force Arm" de Mission Planner), con su propia confirmación aparte — nunca se fuerza por accidente.
- Tras armar, espera 10 segundos (cuenta atrás en el log) y desarma, confirmando también el desarmado.

Por serie directo, o a través de mavlink-router

Admite el parámetro --conexion, con dos valores posibles:

- --conexion serial (por defecto): abre /dev/ttyAMA0 directamente. Si mavlink-router está corriendo como servicio, ya tiene el puerto serie abierto para él solo, así que este modo NO funcionará — hay que pararlo primero (sudo systemctl stop mavlink-router.service), o usar el modo router en su lugar.
- --conexion router: conecta por UDP contra el endpoint de mavlink-router (127.0.0.1:14550 por defecto) en vez de abrir el puerto serie. Es el modo a usar cuando mavlink-router ya está activo y no se quiere parar el servicio solo para hacer esta prueba.

Nota

El COMPID por defecto cambia según el modo (191 en serie, 193 por router) para no coincidir con el de receptor.py (191) si ambos hablan con mavlink-router al mismo tiempo — ver la convención de sysid/compid de la sección 6.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
=====
test-arm-disarm.py – Prueba de armado/desarmado por TELEM3 (serie)
Sistema SAR basado en dron – Raspberry Pi 5
=====

Conecta DIRECTAMENTE al Pixhawk por el puerto serie de TELEM3
(/dev/ttyAMA0 en esta Raspberry Pi – NO /dev/serial0, que en esta placa
apunta a un UART distinto que no está cableado a TELEM3), sin pasar por
mavlink-router ni por UDP. Es una prueba de un único proceso hablando
directo con el autopiloto, pensada para validar el ciclo completo de
armado antes de meter receptor.py/vuelo.py y mavlink-router de por medio.
```

Secuencia:

1. Conecta por serie y espera el heartbeat del autopiloto.
2. Pide confirmación explícita antes de armar (los motores pueden girar de verdad).
3. Arma los motores (respetando los pre-arm checks) y confirma que el autopiloto lo aceptó. Si lo rechaza, muestra el motivo (STATUSTEXT del autopiloto, p. ej. "PreArm: GPS fix") y pregunta si se quiere FORZAR el armado saltándose esos chequeos – equivalente al botón "Force Arm" de Mission Planner, con su propia confirmación explícita aparte.
4. Espera 10 segundos, mostrando una cuenta atrás.
5. Desarma los motores y confirma que el autopiloto lo aceptó.

AVISO DE SEGURIDAD – leer antes de ejecutar:

- Las hélices deben estar DESMONTADAS, o el dron sujeto/atado en un banco de pruebas, tal y como se describe en el documento del proyecto (sección 7, "Procedimiento de prueba y validación").
- Si mavlink-router está corriendo, tiene el puerto serie abierto para él solo y este script no podrá conectar. Pararlo antes:

```
sudo systemctl stop mavlink-router.service
```

 Y volver a arrancarlo después de la prueba:

```
sudo systemctl start mavlink-router.service
```

Uso:

```
python3 test-arm-disarm.py                # serie directo (por defecto)
python3 test-arm-disarm.py --conexion router # vía mavlink-router (UDP)
python3 test-arm-disarm.py --device /dev/ttyAMA0 --baud 57600
python3 test-arm-disarm.py --segundos 15
```

```

python3 test-arm-disarm.py --sin-confirmar # NO recomendado
"""

import sys
import time
import logging
import argparse
from datetime import datetime

from pymavlink import mavutil

# =====
# ARGUMENTOS DE LÍNEA DE COMANDOS
# =====
parser = argparse.ArgumentParser(
    description="Prueba de armado/desarmado del Pixhawk, por serie directo o por mavlink-router")
parser.add_argument("--conexion", choices=["serial", "router"], default="serial",
                    help="'serial' (por defecto): abre el puerto serie directo (--device/--baud). "
                    "'router': conecta contra mavlink-router por UDP (127.0.0.1:14550), "
                    "sin tocar el puerto serie – usa esto si mavlink-router ya está "
                    "corriendo.")
parser.add_argument("--device", default="/dev/ttyAMA0",
                    help="Dispositivo serie, solo con --conexion serial (por defecto: "
                    "/dev/ttyAMA0)")
parser.add_argument("--baud", type=int, default=57600,
                    help="Baudios, solo con --conexion serial (por defecto: 57600, debe coincidir "
                    "con SERIALx_BAUD)")
parser.add_argument("--puerto-router", default="udpin:127.0.0.1:14550",
                    help="Endpoint UDP de mavlink-router, solo con --conexion router "
                    "(por defecto: udpin:127.0.0.1:14550, el mismo que usa receptor.py)")
parser.add_argument("--sysid", type=int, default=1,
                    help="SYSID propio de este proceso (por defecto: 1, igual que el vehículo)")
parser.add_argument("--compid", type=int, default=None,
                    help="COMPID propio de este proceso. Por defecto: 191 con --conexion serial, "
                    "193 con --conexion router (para no coincidir con receptor.py, que ya "
                    "usa "
                    "191 cuando habla contra mavlink-router al mismo tiempo).")
parser.add_argument("--segundos", type=int, default=10,
                    help="Segundos armado antes de desarmar (por defecto: 10)")
parser.add_argument("--sin-confirmar", action="store_true",
                    help="Salta la confirmación manual antes de armar (NO recomendado)")
parser.add_argument("--forzar-directo", action="store_true",
                    help="Si el ARM normal falla, fuerza directamente sin volver a preguntar "
                    "(sigue pidiendo la confirmación 'FORZAR' de seguridad)")
args = parser.parse_args()

if args.compid is None:
    args.compid = 193 if args.conexion == "router" else 191

# =====
# LOGGING – todo por pantalla, con hora exacta de cada paso
# =====
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s.%(msecs)03d [%(levelname)s] %(message)s",
    datefmt="%H:%M:%S",
    stream=sys.stdout,

```

```

)
log = logging.getLogger("test-arm-disarm")

def confirmar_seguridad():
    """Pide confirmación explícita antes de armar. No se puede saltar
    sin pasar --sin-confirmar a propósito.
    """
    log.warning("=" * 70)
    log.warning("AVISO DE SEGURIDAD: este script va a ARMAR los motores.")
    log.warning("Confirma que las hélices están DESMONTADAS, o que el dron")
    log.warning("está firmemente sujeto/atado en un banco de pruebas.")
    log.warning("=" * 70)
    respuesta = input("Escribe exactamente CONFIRMO para continuar: ").strip()
    if respuesta != "CONFIRMO":
        log.error("Confirmación no recibida ('%s'). Abortando sin tocar el autopiloto.",
        respuesta)
        sys.exit(1)
    log.info("Confirmación recibida. Continuando.")

def conectar(conexion, device, baud, puerto_router, sysid, compid):
    """Conecta al autopiloto, o bien por serie directo (TELEM3), o bien
    por UDP contra un endpoint de mavlink-router ya en marcha.
    """
    if conexion == "router":
        destino = puerto_router
        log.info("Conectando a mavlink-router en %s (sysid=%d, compid=%d) ...",
        destino, sysid, compid)
    else:
        destino = device
        log.info("Conectando a %s a %d baudios (sysid=%d, compid=%d) ...",
        destino, baud, sysid, compid)

    try:
        if conexion == "router":
            master = mavutil.mavlink_connection(
                destino,
                source_system=sysid,
                source_component=compid,
            )
        else:
            master = mavutil.mavlink_connection(
                destino,
                baud=baud,
                source_system=sysid,
                source_component=compid,
            )
    except Exception as e:
        log.error("No se ha podido abrir la conexión: %s", e)
        if conexion == "serial":
            log.error("¿No tendrás mavlink-router corriendo? Tiene el puerto serie "
            "abierto para él solo. Prueba: sudo systemctl stop mavlink-router.service")
        else:
            log.error("— o, más simple, relanza este script con --conexion router.")
        log.error("¿Está mavlink-router realmente arrancado? Prueba: "
        "sudo systemctl status mavlink-router.service")
        sys.exit(1)

    log.info("Conexión abierta. Esperando heartbeat del autopiloto (timeout 15s) ...")

```

```

hb = master.wait_heartbeat(timeout=15)
if hb is None:
    log.error("No ha llegado ningún heartbeat en 15 segundos.")
    if conexion == "serial":
        log.error("¿No tendrás mavlink-router corriendo y quedándose con el puerto "
                  "serie? Prueba: sudo systemctl stop mavlink-router.service - o "
                  "relanza este script con --conexion router.")
    else:
        log.error("¿Está mavlink-router corriendo, y con TELEM3 realmente recibiendo "
                  "datos del Pixhawk? Revisa: journalctl -u mavlink-router.service -f")
    log.error("Si mavlink-router ya está descartado, revisa cableado (TX/RX/GND) y "
              "SERIALx_PROTOCOL/BAUD en ArduPilot.")
    sys.exit(1)

log.info("Heartbeat recibido. Autopiloto: sistema=%d, componente=%d",
        master.target_system, master.target_component)
log.info("Tipo de vehículo: %s | Autopiloto: %s",
        mavutil.mavlink.enums["MAV_TYPE"][hb.type].name,
        mavutil.mavlink.enums["MAV_AUTOPILOT"][hb.autopilot].name)
return master

MAGIC_FORZAR_ARM = 21196 # Valor especial que ArduPilot exige en param2 para saltarse
                        # los pre-arm checks (el mismo "Force Arm" de Mission Planner).

def _drenar_mensajes(master, tiempo_max=1):
    """Lee todos los mensajes que lleguen durante 'tiempo_max' segundos.

    Además de refrescar el estado de armado (vía HEARTBEAT), registra en el
    log cualquier STATUSTEXT del autopiloto – es el mecanismo por el que
    ArduPilot explica POR QUÉ rechaza un armado (p. ej. "PreArm: GPS fix").
    """
    limite = time.time() + tiempo_max
    while time.time() < limite:
        msg = master.recv_match(blocking=True, timeout=max(0.0, limite - time.time()))
        if msg is None:
            break
        if msg.get_type() == "STATUSTEXT":
            log.warning(" [STATUSTEXT autopiloto] %s", msg.text)

def esperar_modoy_estado(master, timeout=5):
    """Registra el modo de vuelo actual y el estado de armado antes de tocar nada."""
    master.recv_match(type="HEARTBEAT", blocking=True, timeout=timeout)
    modo = master.flightmode or "DESCONOCIDO"
    armado = master.motors_armed()
    log.info("Estado actual -> modo: %s | armado: %s", modo, armado)
    if armado:
        log.warning("El vehículo YA estaba armado antes de empezar la prueba.")
    return modo, armado

def armar(master, timeout=10):
    """Envía el comando de armado normal (respeta los pre-arm checks) y
    espera confirmación del autopiloto, mostrando cualquier STATUSTEXT
    que explique un posible rechazo.
    """
    log.info("Enviando comando ARM (respetando pre-arm checks) ...")
    t0 = time.time()

```

```

master.arducopter_arm()

log.info("Esperando confirmación de armado (timeout %ds) ...", timeout)
limite = time.time() + timeout
while time.time() < limite and not master.motors_armed():
    _drenar_mensajes(master, tiempo_max=1)

if not master.motors_armed():
    log.error("El autopiloto NO ha aceptado el armado en %d segundos.", timeout)
    log.error("Si arriba no ha aparecido ningún [STATUSTEXT autopiloto] con el "
              "motivo, revisa igualmente ARMING_CHECK y los prerequisites "
              "habituales (GPS fix, calibraciones, failsafes) en Mission Planner.")
    return False

log.info("ARMADO confirmado por el autopiloto (%.2fs).", time.time() - t0)
return True

def armar_forzado(master, timeout=10):
    """Envía el comando de armado FORZADO, saltándose los pre-arm checks
    (equivalente al botón 'Force Arm' de Mission Planner). Usa
    MAV_CMD_COMPONENT_ARM_DISARM con el valor mágico 21196 en param2.
    """
    log.warning("Enviando comando ARM FORZADO (saltando pre-arm checks) ...")
    t0 = time.time()
    master.mav.command_long_send(
        master.target_system,
        master.target_component,
        mavutil.mavlink.MAV_CMD_COMPONENT_ARM_DISARM,
        0,          # confirmation
        1,          # param1: 1 = armar
        MAGIC_FORZAR_ARM, # param2: valor mágico para forzar
        0, 0, 0, 0, 0,
    )

    log.info("Esperando confirmación de armado forzado (timeout %ds) ...", timeout)
    limite = time.time() + timeout
    while time.time() < limite and not master.motors_armed():
        _drenar_mensajes(master, tiempo_max=1)

    if not master.motors_armed():
        log.error("El autopiloto NO ha aceptado el armado NI SIQUIERA FORZADO en "
                  "%d segundos. Esto ya no es un simple pre-arm check – revisa la "
                  "conexión, el failsafe de radio, o si hay algún bloqueo por "
                  "software (safety switch físico, si tu Pixhawk lo tiene).", timeout)
        return False

    log.info("ARMADO FORZADO confirmado por el autopiloto (%.2fs).", time.time() - t0)
    return True

def desarmar(master, timeout=10):
    """Envía el comando de desarmado y espera confirmación del autopiloto."""
    log.info("Enviando comando DISARM ...")
    t0 = time.time()
    master.arducopter_disarm()

    limite = time.time() + timeout
    while time.time() < limite and master.motors_armed():
        _drenar_mensajes(master, tiempo_max=1)

```

```

if master.motors_armed():
    log.error("El autopiloto sigue ARMADO tras %d segundos intentando desarmar.", timeout)
    log.error("¡Atención! Desarma manualmente desde la emisora RC o Mission "
              "Planner de inmediato.")
    return False

log.info("DESARMADO confirmado por el autopiloto (%.2fs).", time.time() - t0)
return True

def cuenta_atras(segundos):
    """Muestra en el log una cuenta atrás mientras el vehículo está armado."""
    log.info("Vehículo armado. Esperando %d segundos antes de desarmar ...", segundos)
    for restante in range(segundos, 0, -1):
        log.info(" ... %d s", restante)
        time.sleep(1)

def confirmar_forzar():
    """Pide una segunda confirmación, más explícita, antes de forzar el
    armado saltándose los pre-arm checks. Nunca se salta con --sin-confirmar
    (ese flag solo afecta a la primera confirmación).
    """
    log.warning("=" * 70)
    log.warning("El comando ARM normal no ha funcionado porque no cumple algún")
    log.warning("chequeo de pre-armado (ARMING_CHECK). ¿Quieres FORZAR el ARM,")
    log.warning("saltándote esos chequeos? Es lo mismo que 'Force Arm' en Mission")
    log.warning("Planner: los motores pueden arrancar aunque algo no esté bien")
    log.warning("(por ejemplo, sin fix GPS o sin calibrar).")
    log.warning("=" * 70)
    respuesta = input("Escribe exactamente FORZAR para continuar, o cualquier otra cosa para
abortar: ").strip()
    return respuesta == "FORZAR"

def main():
    log.info("=" * 70)
    log.info("test-arm-disarm.py - inicio %s", datetime.now().isoformat(timespec="seconds"))
    log.info("=" * 70)

    master = conectar(args.conexion, args.device, args.baud, args.puerto_router,
                      args.sysid, args.compuid)
    esperar_modos_y_estado(master)

    if not args.sin_confirmar:
        confirmar_seguridad()
    else:
        log.warning("Confirmación de seguridad SALTADA (--sin-confirmar). Continuando bajo tu
responsabilidad.")

    armado_ok = armar(master)
    if not armado_ok:
        if args.forzar_directo:
            proceder_forzado = True
        else:
            proceder_forzado = confirmar_forzar()

    if not proceder_forzado:
        log.error("Prueba ABORTADA: no se ha podido armar (normal) y no se ha ")

```

```

        "confirmado el armado forzado. No se intenta desarmar (no "
        "hacía falta, nunca llegó a armar).")
    sys.exit(2)

    armado_ok = armar_forzado(master)
    if not armado_ok:
        log.error("Prueba ABORTADA: tampoco se ha podido armar de forma forzada. "
            "No se intenta desarmar (no hacía falta, nunca llegó a armar).")
        sys.exit(2)

    try:
        cuenta_atras(args.segundos)
    except KeyboardInterrupt:
        log.warning("Interrumpido por el usuario (Ctrl+C). Desarmando inmediatamente.")

    desarmado_ok = desarmar(master)

    log.info("=" * 70)
    if armado_ok and desarmado_ok:
        log.info("PRUEBA COMPLETADA CON ÉXITO: armado y desarmado confirmados por el
autopiloto.")
    else:
        log.error("PRUEBA COMPLETADA CON ERRORES. Revisa los mensajes anteriores.")
    log.info("=" * 70)

if __name__ == "__main__":
    main()

```

C.4 test-estado.py — panel en vivo de batería, GPS y RC

Un panel que se refresca cada segundo en la terminal, útil para vigilar el estado del vehículo mientras se ajustan cosas (calibración de radio, posicionamiento para fix GPS, etc.) sin tener que interpretar mensajes MAVLink sueltos a mano.

- BATERÍA: voltaje, corriente, porcentaje restante, con aviso si baja del 20%.
- GPS: tipo de fix, satélites visibles, HDOP, posición, con aviso si todavía no hay fix 3D.
- RC: si el receptor está presente y sano, valores en crudo de los 8 primeros canales, RSSI (con la aclaración de que muchos receptores PPM/SBUS no lo reportan, sin que eso sea un fallo), y un aviso heurístico si el throttle no parece estar al mínimo.
- PRE-ARM: los mensajes STATUSTEXT reales del autopiloto explicando qué falla para poder armar. El script pide activamente este re-chequeo cada pocos segundos (MAV_CMD_RUN_PREARM_CHECKS) en vez de esperar al ciclo automático de ~30s de ArduPilot — es la única fuente fiable de este dato, ya que ArduPilot nunca ha implementado el bit de "listo para armar" del estándar MAVLink en SYS_STATUS (issue conocido y abierto en su GitHub, #13534).

Admite el mismo parámetro `--conexion serial|router` que `test-arm-serial.py`, con la misma lógica: usar router si mavlink-router ya está corriendo y no se quiere parar el servicio.

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
=====

```

test-estado.py – Panel en vivo: batería, GPS y RC del Pixhawk
Sistema SAR basado en dron – Raspberry Pi 5

Conecta al autopiloto (por serie directo o por mavlink-router, igual que test-arm-disarm.py) y muestra un panel que se refresca cada segundo con:

- BATERÍA: voltaje, corriente, porcentaje restante.
- GPS: tipo de fix, satélites visibles, HDOP, posición.
- RC: si el receptor está presente y sano, valores en crudo de los canales, y un aviso heurístico de si el throttle parece estar en una posición que bloquearía el armado (no al mínimo).
- PRE-ARM: los mensajes STATUSTEXT reales que envía ArduPilot explicando qué falla para poder armar (p. ej. "PreArm: GPS: no fix"). El script pide activamente estos mensajes cada pocos segundos (MAV_CMD_RUN_PREARM_CHECKS), porque ArduPilot NO expone un booleano fiable de "listo para armar" en SYS_STATUS – es un hueco conocido y nunca implementado del firmware (ver issue ArduPilot/ardupilot #13534). Esta es la única fuente real de ese dato por MAVLink.

No modifica nada del vehículo salvo pedir el re-chequeo de pre-arm (MAV_CMD_RUN_PREARM_CHECKS), que no arma ni cambia ningún parámetro: es de solo lectura para todo lo demás.

AVISO sobre el chequeo de RC: es una comprobación HEURÍSTICA basada en el valor en crudo del canal de throttle (por defecto, canal 3). No sustituye a los pre-arm checks reales de ArduPilot – para ver el motivo EXACTO de un rechazo de armado, usa test-arm-disarm.py, que captura los mensajes STATUSTEXT del propio autopiloto.

Uso:

```
python3 test-estado.py           # serie directo (por defecto)
python3 test-estado.py --conexion router  # vía mavlink-router (UDP)
python3 test-estado.py --canal-throttle 3  # cambiar el canal si RCMAP_THROTTLE no es el 3
(CTRL+C para salir)
```

```
import sys
import time
import shutil
import logging
import argparse
```

```
from pymavlink import mavutil
```

```
# =====
# ARGUMENTOS DE LÍNEA DE COMANDOS
# =====
parser = argparse.ArgumentParser(
    description="Panel en vivo de batería, GPS y RC del Pixhawk, por serie o por mavlink-router")
parser.add_argument("--conexion", choices=["serial", "router"], default="serial",
    help="'serial' (por defecto): puerto serie directo (--device/--baud). "
    "'router': UDP contra mavlink-router (127.0.0.1:14550), sin tocar "
    "el puerto serie – usa esto si mavlink-router ya está corriendo.")
parser.add_argument("--device", default="/dev/ttyAMA0",
    help="Dispositivo serie, solo con --conexion serial (por defecto: "
    "/dev/ttyAMA0)")
```

```

parser.add_argument("--baud", type=int, default=57600,
                    help="Baudios, solo con --conexion serial (por defecto: 57600)")
parser.add_argument("--puerto-router", default="udpin:127.0.0.1:14550",
                    help="Endpoint UDP de mavlink-router, solo con --conexion router "
                        "(por defecto: udpin:127.0.0.1:14550)")
parser.add_argument("--sysid", type=int, default=1,
                    help="SYSID propio de este proceso (por defecto: 1, igual que el vehículo)")
parser.add_argument("--compid", type=int, default=None,
                    help="COMPID propio de este proceso. Por defecto: 191 con --conexion serial, "
                        "\"194 con --conexion router (para no coincidir con receptor.py=191 ni "
                        "\"vuelo.py=192 si están hablando con mavlink-router a la vez).")
parser.add_argument("--canal-throttle", type=int, default=3,
                    help="Número de canal RC que corresponde al throttle (por defecto: 3, "
                        "\"el estándar de ArduPilot; ajústalo si tu RCMAP_THROTTLE es distinto)")
parser.add_argument("--frecuencia", type=float, default=1.0,
                    help="Refresco del panel en segundos (por defecto: 1.0)")
args = parser.parse_args()

if args.compid is None:
    args.compid = 194 if args.conexion == "router" else 191

# =====
# LOGGING (solo para errores de arranque; el panel usa print directo)
# =====
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s.%(msecs)03d [%(levelname)s] %(message)s",
    datefmt="%H:%M:%S",
    stream=sys.stderr,
)
log = logging.getLogger("test-estado")

# =====
# CONEXIÓN
# =====
def conectar():
    if args.conexion == "router":
        destino = args.puerto_router
        log.info("Conectando a mavlink-router en %s (sysid=%d, compid=%d) ...",
                destino, args.sysid, args.compid)
        kwargs = {}
    else:
        destino = args.device
        log.info("Conectando a %s a %d baudios (sysid=%d, compid=%d) ...",
                destino, args.baud, args.sysid, args.compid)
        kwargs = {"baud": args.baud}

    try:
        master = mavutil.mavlink_connection(
            destino, source_system=args.sysid, source_component=args.compid, **kwargs)
    except Exception as e:
        log.error("No se ha podido abrir la conexión: %s", e)
        if args.conexion == "serial":
            log.error("¿No tendrás mavlink-router corriendo? Prueba --conexion router, "
                    "o sudo systemctl stop mavlink-router.service")
        else:
            log.error("¿Está mavlink-router realmente arrancado? "
                    "sudo systemctl status mavlink-router.service")

```

```

sys.exit(1)

log.info("Conexión abierta. Esperando heartbeat (timeout 15s) ...")
hb = master.wait_heartbeat(timeout=15)
if hb is None:
    log.error("No ha llegado ningún heartbeat en 15 segundos.")
    if args.conexion == "serial":
        log.error("¿No tendrás mavlink-router corriendo y quedándose con el puerto? "
                  "Prueba --conexion router, o para mavlink-router primero.")
    else:
        log.error("¿mavlink-router está corriendo y TELEM3 recibe datos del Pixhawk? "
                  "Revisa: journalctl -u mavlink-router.service -f")
    sys.exit(1)

log.info("Heartbeat recibido: sistema=%d, componente=%d",
        master.target_system, master.target_component)

# Pide explícitamente todos los streams de datos a 4 Hz – sin esto,
# el autopiloto puede no enviar por su cuenta todo lo que necesitamos.
master.mav.request_data_stream_send(
    master.target_system, master.target_component,
    mavutil.mavlink.MAV_DATA_STREAM_ALL, 4, 1)

return master

def pedir_prearm_checks(master):
    """Pide al autopiloto que vuelva a evaluar y enviar por STATUSTEXT el
    motivo de cualquier fallo de pre-arm, sin esperar a su ciclo automático
    de ~30s. ArduPilot no expone un booleano fiable de "listo para armar"
    en SYS_STATUS (bug conocido, nunca implementado) – los STATUSTEXT
    "PreArm: ..." son la única fuente real de este dato.
    """
    master.mav.command_long_send(
        master.target_system, master.target_component,
        mavutil.mavlink.MAV_CMD_RUN_PREARM_CHECKS,
        0, 0, 0, 0, 0, 0, 0, 0, 0,
    )

# =====
# INTERPRETACIÓN DE MENSAJES
# =====
FIX_TYPE_NOMBRES = {
    0: "SIN GPS", 1: "SIN FIX", 2: "FIX 2D", 3: "FIX 3D",
    4: "DGPS", 5: "RTK FLOAT", 6: "RTK FIJO", 7: "ESTÁTICO", 8: "PPP",
}

SENSOR_RC_RECEIVER = mavutil.mavlink.MAV_SYS_STATUS_SENSOR_RC_RECEIVER

def resumen_bateria(msgs):
    sys_status = msgs.get("SYS_STATUS")
    bat = msgs.get("BATTERY_STATUS")

    if sys_status is None and bat is None:
        return [" (todavía sin datos de batería)"]

    lineas = []
    if sys_status is not None:

```

```

    v = sys_status.voltage_battery
    i = sys_status.current_battery
    pct = sys_status.battery_remaining
    v_txt = f"{v/1000:.2f} V" if v != 65535 else "N/D"
    i_txt = f"{i/100:.1f} A" if i != -1 else "N/D"
    pct_txt = f"{pct} %" if pct != -1 else "N/D"
    lineas.append(f" Voltaje: {v_txt} Corriente: {i_txt} Restante: {pct_txt}")
    if pct != -1 and pct < 20:
        lineas.append(" ⚠ Batería por debajo del 20% – no volar.")
    return lineas

def resumen_gps(msgs):
    gps = msgs.get("GPS_RAW_INT")
    if gps is None:
        return [" (todavía sin datos de GPS)"]

    fix_txt = FIX_TYPE_NOMBRES.get(gps.fix_type, f"desconocido ({gps.fix_type})")
    sats = gps.satellites_visible if gps.satellites_visible != 255 else "N/D"
    hdop = gps.eph / 100.0 if gps.eph != 65535 else None
    lat = gps.lat / 1e7
    lon = gps.lon / 1e7

    lineas = [f" Fix: {fix_txt} Satélites: {sats}" +
              (f" HDOP: {hdop:.2f}" if hdop is not None else "")]
    if gps.fix_type >= 3:
        lineas.append(f" Posición: {lat:.7f}, {lon:.7f}")
    else:
        lineas.append(" ⚠ Sin fix 3D todavía – ArduPilot normalmente no arma sin GPS "
                      "(salvo que ARMING_CHECK lo excluya).")
    return lineas

def resumen_rc(msgs, canal_throttle):
    sys_status = msgs.get("SYS_STATUS")
    rc = msgs.get("RC_CHANNELS")

    lineas = []

    if sys_status is not None:
        presente = bool(sys_status.onboard_control_sensors_present & SENSOR_RC_RECEIVER)
        sano = bool(sys_status.onboard_control_sensors_health & SENSOR_RC_RECEIVER)
        if not presente:
            lineas.append(" ⚠ El autopiloto no reporta ningún receptor RC presente.")
        elif not sano:
            lineas.append(" ⚠ Receptor RC presente pero marcado como NO SANO "
                          "(posible failsafe o pérdida de señal).")
        else:
            lineas.append(" Receptor RC: presente y sano.")

    if rc is None:
        lineas.append(" (todavía sin datos de canales RC)")
        return lineas

    canales = [rc.chan1_raw, rc.chan2_raw, rc.chan3_raw, rc.chan4_raw,
               rc.chan5_raw, rc.chan6_raw, rc.chan7_raw, rc.chan8_raw]
    etiquetas = ["ROLL", "PITCH", "THR", "YAW", "AUX1", "AUX2", "AUX3", "AUX4"]
    fila = " " + " ".join(f"{et}={val}" for et, val in zip(etiquetas, canales))
    lineas.append(fila)

```

```

rssi = rc.rssi
if rssi == 255:
    lineas.append(" RSSI: no reportado por este receptor/protocolo (normal en muchos "
                  "receptores PPM/SBUS sin RSSI_TYPE configurado – no es un problema si "
                  "los valores de canal de arriba cambian con la emisora).")
elif rssi == 0:
    lineas.append(" ⚠ RSSI en 0 – si además los canales de arriba no se mueven al mover "
                  "la emisora, sí podría ser pérdida de señal real. Si los canales cambian "
                  "con normalidad, probablemente sea el mismo caso de \"no reportado\".")
else:
    lineas.append(f" RSSI: {rssi}")

idx = canal_throttle - 1
if 0 <= idx < len(canales):
    val_thr = canales[idx]
    if val_thr == 0:
        lineas.append(f" ⚠ Canal {canal_throttle} (throttle) en 0 – no hay señal RC "
                      "real en ese canal, revisa --canal-throttle.")
    elif val_thr > 1200:
        lineas.append(f" ⚠ Throttle (canal {canal_throttle}) = {val_thr}, no parece "
                      "estar al mínimo – esto suele BLOQUEAR el armado en ArduPilot. "
                      "Baja el stick de gas del todo.")
    else:
        lineas.append(f" Throttle (canal {canal_throttle}) = {val_thr} – parece al "
                      "mínimo, correcto para armar.")

return lineas

def resumen_general(master, msgs):
    modo = master.flightmode or "DESCONOCIDO"
    armado = master.motors_armed()
    lineas = [f" Modo: {modo} Armado: {'SÍ' if armado else 'no'}"]
    return lineas

# =====
# BUCLE PRINCIPAL – panel que se refresca
# =====
def limpiar_pantalla():
    print("\033[H\033[J", end="")

def main():
    master = conectar()
    print() # deja el log de conexión visible antes del primer refresco

    prearm_msgs = [] # últimos STATUSTEXT relacionados con pre-arm
    ultima_peticion = 0.0
    INTERVALO_PETICION = 6.0 # cada cuántos segundos forzar un re-chequeo

    try:
        while True:
            ahora = time.time()
            if ahora - ultima_peticion >= INTERVALO_PETICION:
                pedir_pream_checks(master)
                ultima_peticion = ahora

            # Drena todo lo recibido en esta vuelta, sin bloquear, y de paso
            # captura cualquier STATUSTEXT que llegue (no queda en .messages,

```

```

# solo se ve el último de cada tipo, así que hay que cogerlo aquí).
while True:
    msg = master.recv_match(blocking=False)
    if msg is None:
        break
    if msg.get_type() == "STATUSTEXT":
        texto = msg.text.strip()
        if texto and (not prearm_msgs or prearm_msgs[-1] != texto):
            prearm_msgs.append(texto)
            prearm_msgs[:] = prearm_msgs[-5:] # solo los 5 últimos

msgs = master.messages

ancho = shutil.get_terminal_size((80, 24)).columns
limpiar_pantalla()
print("=" * ancho)
print(" ESTADO DEL PIXHAWK – Ctrl+C para salir".ljust(ancho))
print("=" * ancho)

print("\nGENERAL")
for l in resumen_general(master, msgs):
    print(l)

print("\nBATERÍA")
for l in resumen_bateria(msgs):
    print(l)

print("\nGPS")
for l in resumen_gps(msgs):
    print(l)

print("\nRC")
for l in resumen_rc(msgs, args.canal_throttle):
    print(l)

print("\nPRE-ARM (mensajes del autopiloto)")
if prearm_msgs:
    for texto in prearm_msgs:
        print(f" {texto}")
else:
    print(" (sin mensajes todavía – si no aparece nada en unos segundos, "
          "puede que ya esté listo para armar, o que no reciba STATUSTEXT)")

print("\n" + "=" * ancho)
time.sleep(args.frecuencia)

except KeyboardInterrupt:
    print("\nDetenido por el usuario.")

if __name__ == "__main__":
    main()

```